

Inference and Serving Engineering - Crash Pack

This is the compressed volume for Track 03. It covers everything that decides whether a serving stack survives production, reduced to decision rules, memorized numbers, traps, and drills. It assumes the base material (the generate loop, continuous batching, the KV cache, quantization basics) is familiar. What it adds is the judgment layer: which choice to make, what number to check, and what breaks when you choose wrong.

Three questions organize the whole track, and this pack. How fast is each answer? How many answers per GPU per second? What does each good answer cost? Every rule below moves at least one of those three.

Worked-number policy. Every number here is an illustrative example built from public spec-sheet and listing values (September 2026), not a measured benchmark. The method is the point: redo the math with your own hardware prices and measured throughput before spending money.

Reference hardware: H100 SXM at 989 TFLOPS BF16 dense (1979 TFLOPS FP8), 3.35 TB/s HBM. BF16 is bfloat16, a 16-bit float format. FP8 is 8-bit floating point. On-demand H100 costs around \$3/hr. The reference model is 70B-class with 80 layers and 8 KV heads.

1. Decision rules

1.1 Batching strategy selection

Three words this whole pack leans on. A token is the unit the model reads and writes: roughly a word piece (about 4 English characters on average). The prompt is the user's input text. Decoding generates the answer one token at a time.

Batching is how you keep the GPU busy instead of waiting. The ladder climbs from one request at a time to a batch that refills itself mid-flight.

Prefill and decode want different things

Prefill runs once per request. Decode runs once per token.

Prefill

one full pass over 2k tokens: 0.29 s

compute-bound: FLOPS-hungry, fully parallel

Decode

t1

t2

t3

t4

t5

t6

t7

t8

memory-bound: 0.042 s per token, full 140 GB weight read each step

Same weights, wildly different reuse.

Prefill reuses the weights across 2048 tokens in one pass.

Decode reads the same 140 GB for a single token, again and again.

Figure 1.1. Prefill and decode: two phases, two bottlenecks. Prefill is one parallel compute-bound pass. Decode is a memory-bound step per token, reading the full weight matrix each time.

Step 1: one request at a time

The GPU runs one decode step, produces one token, then waits for the next request. The hardware bills by the second whether it works or waits, so most of a \$30,000 GPU sits idle. Nobody serves like this in production. It is the baseline the rest of the ladder improves on.

Step 1: one request at a time

The GPU works, then waits. Expensive hardware sits idle between requests.

request A: decode steps

idle

request B: decode steps

idle

Utilization is terrible: the GPU bills by the second whether it works or waits.

One request at a time. Blue bars are work; dashed red boxes are idle GPU time you still pay for.

Step 2: static batching, and the straggler tax

Group N requests, run them together, and wait until the longest finishes before starting the next group. Throughput rises, but every request waits for the slowest one in its group. One 128k-output request holds the whole batch hostage. That waiting has a name: the **straggler tax**. Static batching is acceptable only for offline bulk jobs with uniform lengths, where the tax is small because every request is the same length.

Static batching (only for uniform offline work):

```

step 1: [ req A ..... ]   short request B waits
step 1: [ req B ..... ]   for the longest request
step 2: [ req C ..][ req D ..] in the batch. Straggler tax.

```

Continuous batching (default for online traffic):

```

step 1: [A][B][C][D]
step 2: [A][_][C][D]  <- B finished, E admitted instantly
step 3: [A][E][C][D]

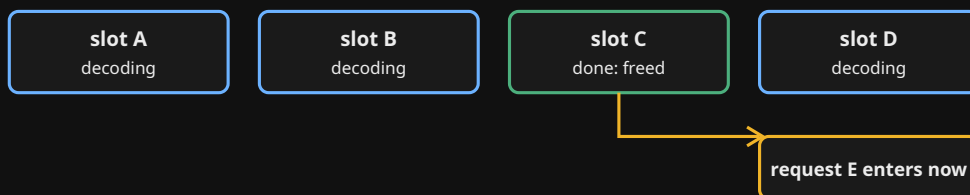
```

Step 3: continuous (in-flight) batching

Admit a new request the moment any slot frees, at single-decode-step granularity. No request waits for another to finish. This is the default in every serious engine for all online serving: chat, agents, and mixed lengths.

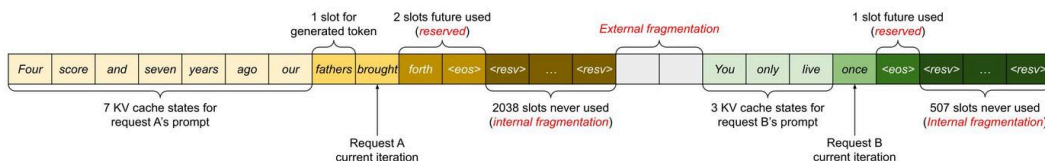
Step 3: continuous batching

A slot frees mid-flight. The next request enters on the very next decode step.



Continuous batching. Slot C finishes; request E takes its place immediately. Nothing waits for the slowest request.

Continuous batching only works if the engine grants and reclaims memory per request instead of reserving it up front. The old way reserved the maximum length for every request, and most of it sat wasted:



What PagedAttention fixes. The old way reserves the maximum possible length for every request up front: slots sit never-used inside each reservation (internal fragmentation), plus gaps between reservations (external fragmentation). Figure: Kwon et al., SOSP 2023.

Continuous batching: slots refill mid-flight

A slot frees mid-flight. The next request enters on the very next decode step.



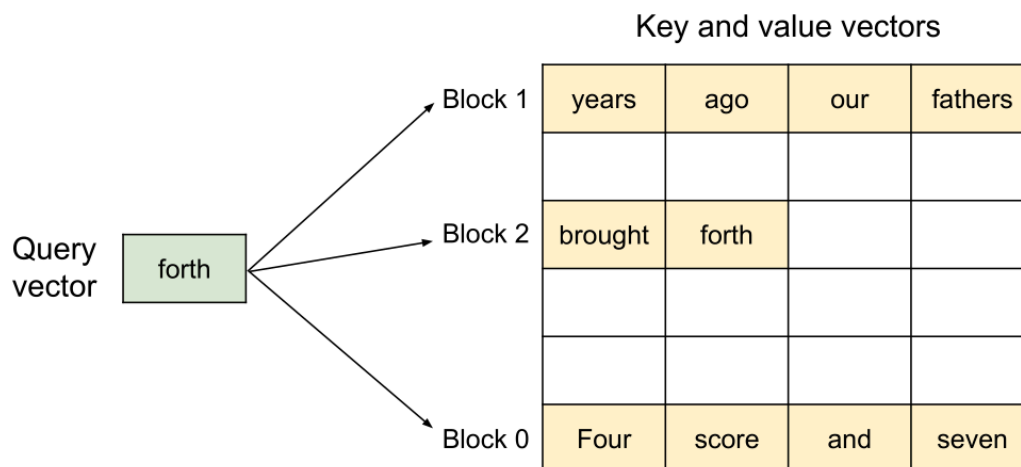
Slot 1: request A finishes at t3, request B enters at t4. No waiting for the batch.

Slot 3: three short requests share one slot across the same seven steps.

Slot 4: idle at t1, then request G starts immediately. Idle time is the enemy.

Figure 1.2. Continuous batching in seven steps. Slot 1 frees at t3 and refills at t4. Short requests share slots. Idle time is the enemy.

The fix is **PagedAttention**: the KV cache lives in small blocks, allocated on demand, tracked by a block table. No more giant reserved slots, so the memory waste drops to near zero:



The query for 'forth' reads blocks 1, 2, and 0, which sit anywhere in memory. This is the whole trick: no more reserving one giant contiguous slot per request, so memory waste drops to near zero. Figure: Kwon et al., PagedAttention (SOSP 2023).

Step 4: mixed traffic needs chunked prefill

Prefill is the phase that reads the whole prompt in one pass before generating any token. Long prompts mixed with short chat break the smoothness: one giant prefill stalls every decode step behind it. **Chunked prefill** splits each prompt into fixed-size chunks (a few

hundred tokens) and interleaves one chunk per decode step.

Two latency terms this pack uses everywhere. **TTFT** (time to first token) is queue wait plus prefill time: this is responsiveness. **TPOT** (time per output token) is the average decode step time: this is the streaming reading speed. Chunked prefill raises TTFT a little and keeps TPOT smooth, because no single step gets stalled by a giant prefill.

The chunk-size knob: bigger chunks lower TTFT but make decode steps lumpier; smaller chunks smooth TPOT but stretch TTFT. Tune it against your TTFT and TPOT budgets, not by feel.

Batching set how you use the GPUs. Quantization changes what fits on them: it shrinks the bytes per weight (and per KV entry), which buys memory, bandwidth, or both.

1.2 Quantization method selection

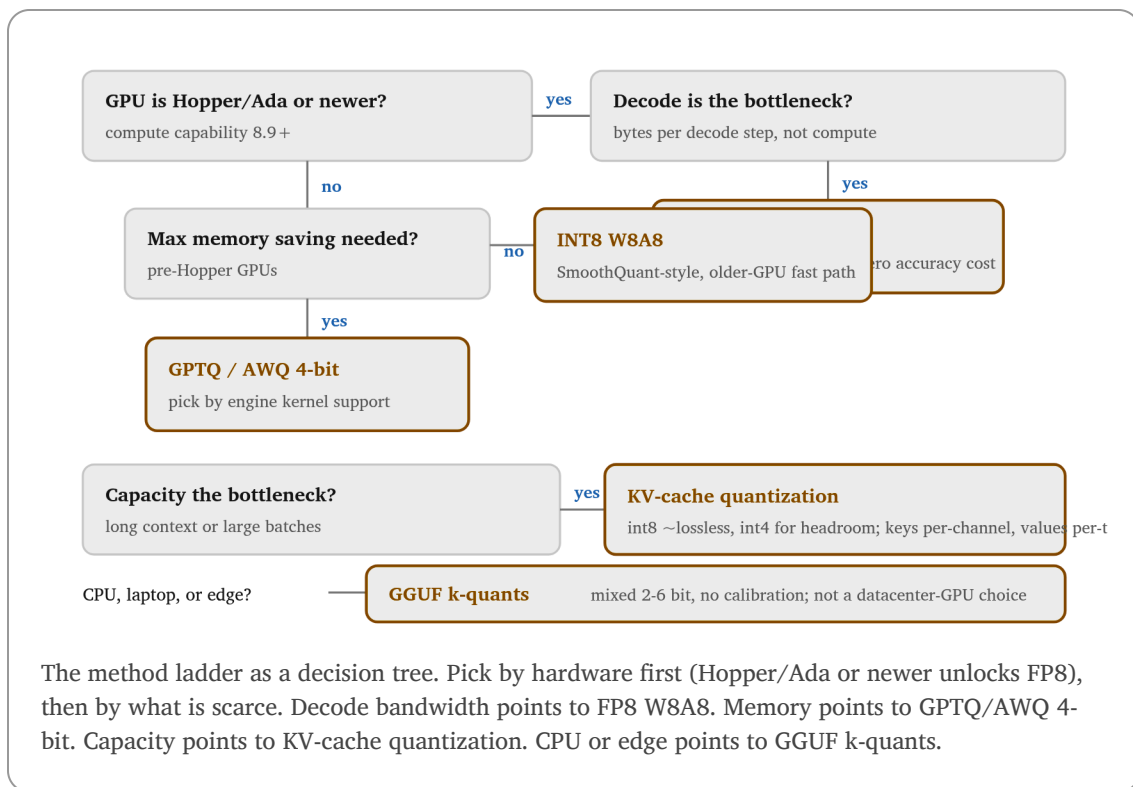
Quantization shrinks the bytes per weight (and per KV entry), which buys memory, bandwidth, or both. The ladder: one selection rule, then the full table, then the three ways it breaks in production.

Step 1: the selection rule

Pick by hardware first, then by what is scarce: memory, throughput, or capacity. **W8A8** means 8-bit weights and 8-bit activations: every number in the matmul (matrix multiplication, the core operation of a neural network) is 8 bits, so each decode step moves half the bytes.

Step 2: the table, the full picture

Situation	Method	Why it wins
Hopper/Ada fleet (compute capability 8.9+, i.e. Hopper/Ada and newer), decode is the bottleneck	FP8 W8A8	Native hardware support, near-zero accuracy cost on most models, halves bytes per decode step. Dynamic scales to try it, static scales (calibrated offline) to ship it.
Pre-Hopper GPUs, maximum memory saving	GPTQ or AWQ 4-bit	Both are methods that compress weights to 4 bits by choosing rounding cleverly, and both reach similar 4-bit accuracy. The choice is a kernel question: pick whichever your engine's fused kernels support best (Marlin is one fast 4-bit kernel; the point stands for any fused kernel), and benchmark both on your eval set.
Pre-Hopper, INT8 is the fast path	INT8 W8A8 (SmoothQuant-style)	Uses INT8 tensor cores (the GPU's dedicated matrix-multiply units) on older GPUs where FP8 does not exist.
Long context or large batches, capacity is the bottleneck	KV-cache quantization	int8 KV is nearly lossless in current engines; int4 buys large headroom. This buys batch size, not step speed. Keys per-channel, values per-token.
CPU, laptop, or edge	GGUF k-quants	GGUF is a file format for running models on CPU; k-quants are its mixed 2-to-6-bit variants. No calibration. Not a datacenter-GPU choice.



Step 3: the three production gotchas

- **Eval harness first.** Never quantize without running your task evals first. Perplexity (the model's average surprise per token, lower is better) can look unchanged while math and code accuracy fall off a cliff. Section 2.6 quantifies it: a 4-bit model with flat perplexity and a 6-point drop on math tasks. Evaluate on the tasks your users run.
- **Re-calibrate after every fine-tune, merge, or distillation.** Quantization scales fit one checkpoint. They are not portable to the next.
- **Test both traffic shapes.** Activation quantization bites hardest in prefill. A W8A8 setup that looks lossless on decode-heavy chat can regress on prefill-heavy long-document work.

The FP8 deployment modes, as a decision (dynamic to try it, static to ship it):

```
# fp8_select.py - the two FP8 deployment modes, as a decision.
# WHAT: dynamic FP8 needs no calibration; static FP8 precomputes scales.
# WHY: dynamic per-step scale computation costs speed. Static removes it.
# WHAT BREAKS: benchmarking dynamic FP8 and concluding "FP8 is slow".
# You measured the wrong mode. Ship static; use dynamic only to try it.

def fp8_mode(hardware_cc, production):
    # hardware_cc: GPU compute capability, e.g. 9.0 for H100.
    # Returns the recommended FP8 path.
    if hardware_cc < 8.9:
        # No W8A8 here. Ampere gets FP8 weights only via Marlin kernels.
        return "W8A16 weight-only (Marlin) or GPTQ/AWQ 4-bit"
    if production:
        return "static FP8: calibrate once with llm-compressor, serve fixed
scales"
    return "dynamic FP8: --quantization fp8, no prep, accept the scale
overhead"

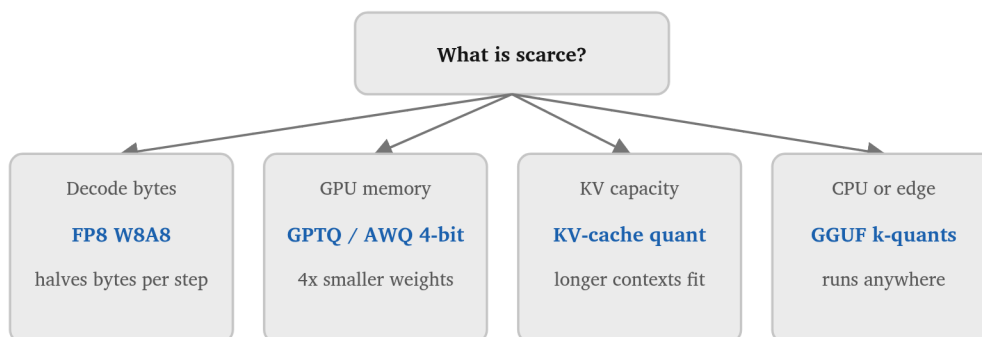
for cc, prod in [(9.0, False), (9.0, True), (8.0, True)]:
    print(f"cc={cc}, production={prod}: {fp8_mode(cc, prod)}")
```

```
cc=9.0, production=False: dynamic FP8: --quantization fp8, no prep, accept the
scale overhead
cc=9.0, production=True: static FP8: calibrate once with llm-compressor, serve
fixed scales
cc=8.0, production=True: W8A16 weight-only (Marlin) or GPTQ/AWQ 4-bit
```

Batching and quantization tuned one box. Disaggregation splits the work across boxes by phase, because prefill and decode want different hardware.

Pick the quantization method by what is scarce

Hardware first: Hopper or Ada or newer unlocks FP8.



Scales fit one checkpoint. They are not portable to the next.

Test both traffic shapes: activation quantization bites hardest in prefill.

Figure 1.3. The quantization decision tree. Pick by what is scarce: decode bytes, GPU memory, KV capacity, or CPU/edge. Hardware first: Hopper or Ada unlocks FP8.

1.3 Disaggregated prefill/decode: the decision rule

The ladder for this decision has three steps. Step 1: why the two phases want different hardware. Step 2: what disaggregation does about it. Step 3: the conditions plus the quantitative gate that decide it.

Step 1: prefill and decode want different things

Prefill is compute-bound and wants FLOPS. Decode is memory-bound and wants bandwidth and capacity. On one shared pool they fight: long prefills stall running decodes.

Step 2: what disaggregation does

Disaggregation puts prefill and decode on separate GPU pools and ships the KV cache between them over RDMA (direct memory-to-memory transfer over the network, no CPU involved). It wins only when the transfer tax is smaller than the interference it removes.

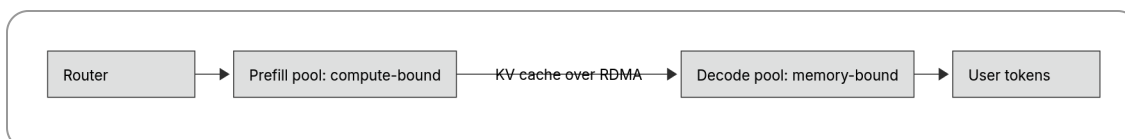
Step 3: the decision

Disaggregate when at least two of these hold:

1. TTFT SLO is tight (under ~ 1 s) and prefill/decode interference is the *measured* cause of violations, not queueing. (An SLO is a service-level objective: a promised number, like "p99 TTFT under 1 s". p99 is the latency 99% of requests beat.)
2. Prompts are long ($8k+$): prefill dominates the request's GPU time, so isolating it moves the needle.
3. Decode batches are small because prefill hogs KV memory: the shared pool forces an unhappy compromise.
4. The interconnect between pools is fast: same-node NVLink, or RDMA with measured transfer well under the TTFT budget.

Do not disaggregate when prompts are short (transfer overhead dominates). Do not split when one pool would idle most of the day (two half-empty pools cost more than one full one). Do not split when the team cannot operate the transfer layer reliably. **The default is a shared pool with chunked prefill.** Disaggregation is the optimization you graduate to with measurements in hand.

The quantitative gate (worked in code in section 2.7) has two conditions. The transfer must stay under a quarter of the TTFT budget. *And* the transfer must be cheaper than the interference it removes. A 2k-token prefill chunk stalls running decodes ~ 0.3 s on a shared GPU; if the transfer costs 6 ms, disaggregation wins big. If the transfer costs 200 ms and the interference was 50 ms, it loses.

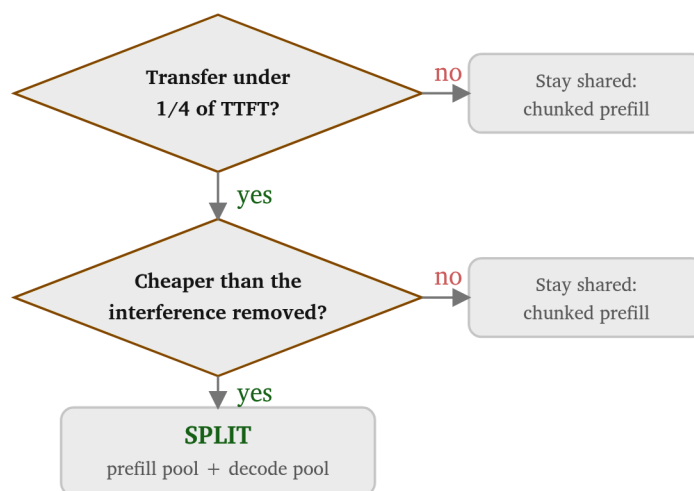


How to read this diagram

1. The router sends each prompt to the prefill pool only. That pool runs the full forward pass over the prompt and builds the KV cache.
2. The middle arrow is the price of admission: the KV cache ships to the decode pool over RDMA (Mooncake/NIXL style). Its cost in milliseconds is the transfer math in section 2.7.
3. The decode pool never recomputes the prompt. It streams tokens, freed from prefill stalls, which is where the ~30% lower time-between-tokens comes from in published setups.
4. Teaching point: if the middle arrow is slow (cross-rack at long context), the whole diagram loses to a shared pool. Measure the arrow before you build the pools.

The disaggregation gate: two numbers decide

Disaggregation is the optimization you graduate to with measurements in hand.



Do not split when prompts are short, when one pool would idle, or when the team cannot run the transfer layer.

Figure 1.4. The disaggregation gate. Split only if the transfer stays under a quarter of the TTFT budget and costs less than the interference it removes. Otherwise stay shared with chunked prefill.

How to read this diagram

1. Start at the top diamond: measure the KV transfer for your prompt lengths and compare it to a quarter of your TTFT budget. If it fails, stay shared.
2. Pass the first gate, then price the interference: how much decode throughput does prefill steal on a shared GPU? If the transfer costs more than the stolen throughput, stay shared.
3. Pass both gates and split: a prefill pool and a decode pool, sized independently from the traffic mix.
4. What it is: a two-number decision rule. Why it exists: disaggregation trades interference for transfer bytes, and the trade only wins sometimes. What breaks: short prompts (transfer dominates), idle pools (two half-empty pools cost more than one full one), an unreliable transfer layer.

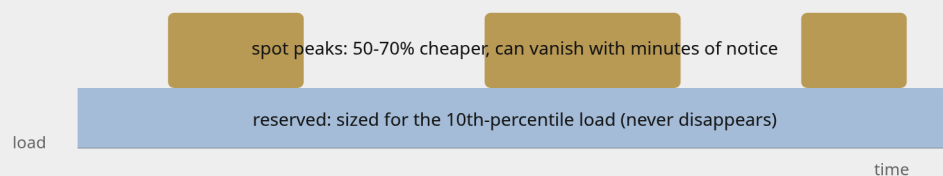
The single-box decisions are made. Now the fleet: which GPUs to rent, at which price, and how to mix them.

1.4 Spot vs reserved fleet mix

Cloud GPUs come in two prices. Reserved capacity costs the sticker price and stays yours. Spot capacity costs 50-70% less, and the provider can take it back with minutes of notice. The decision is a mix, set by preemption tolerance. Climb it in four steps:

Step 1: baseline on reserved

- **Baseline traffic on reserved.** The traffic floor that is always there belongs on capacity that never disappears. Size reserved for the **10th-percentile load**, not the mean.
- **Step 2: peaks on spot.** Traffic above baseline scales onto spot with an autoscaler. When spot vanishes, degrade gracefully: queue, shed low-priority traffic, or fall back to a smaller model through a cascade.

Spot vs reserved: size reserved for the floor, peaks on spot

When spot vanishes: queue, shed low-priority traffic, or cascade to a smaller model.

Spot vs reserved mix. Reserved covers the load floor that is always there; peaks ride spot at 50-70% off. When spot vanishes, queue, shed, or cascade.

- **Step 3: prefill is more spot-friendly than decode.** A prefill is short and cheap to redo. Decode workers hold KV state for in-flight requests, so a preemption kills those requests outright. Keep spot to a fraction where the retry cost stays small.
- **Step 4: check the math.** Spot wins the breakeven for anything but the strictest latency SLOs. The worked discount numbers live in section 2.10.

What actually kills spot savings is not preemption. It is **slow autoscaling** that leaves spot GPUs idle while reserved GPUs queue.

You pay for the fleet. Routing decides which requests each part of the fleet sees, and cascades let a cheap model handle the easy ones.

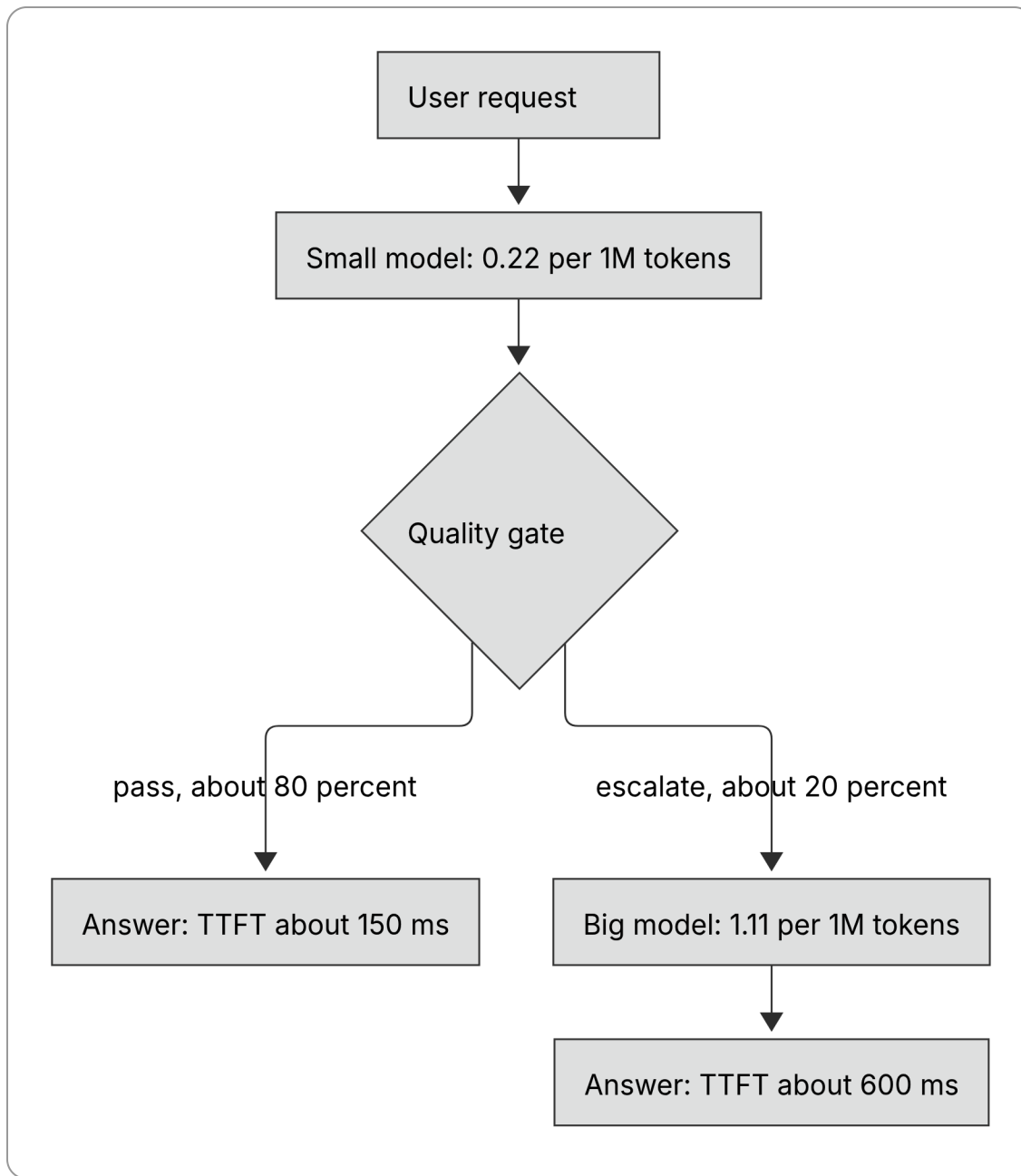
1.5 Routing and cascade rules

A cascade serves every request with a small cheap model first and checks the answer with a quality gate. It escalates only the hard ones to the big model. A router is the same idea with more tiers or per-request model choice. Three steps:

Step 1: the small model answers everything

Every request enters the small model first and pays its cost. There is no way to skip it.

Step 2: the gate decides

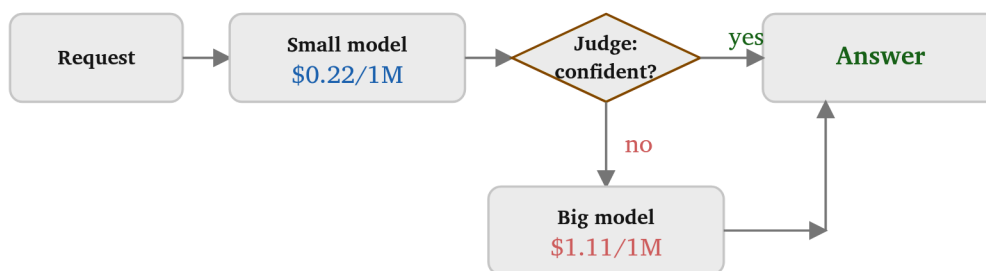


How to read this diagram

1. Every request enters the small 8B model first (\$0.22 per 1M tokens). There is no way to skip this cost.
2. The quality gate inspects the answer: a confidence threshold (cheap, but confidence is not correctness), a judge model (accurate, costs one more inference call), or a rule like "route math to the big model" (zero cost, blind to difficulty within a type).
3. About 80% pass and return immediately at ~150 ms TTFT. These users feel the small model's speed.
4. About 20% escalate and pay the escalation tax: small + gate + big, ~600 ms TTFT. Escalated requests always pay twice.
5. Teaching point: the gate is the load-bearing piece. A sloppy gate either escalates too much (kills the savings) or accepts bad answers (kills quality while the token-cost metric looks great).

Cascade: the small model answers everything first

A router is the same idea with more tiers or per-request model choice.



The escalation tax.

Escalated requests pay both models: $0.22 + 0.05 + 0.20 \times 1.11 = \$0.49/1M$. Track the escalation rate like a budget.

Figure 1.5. The cascade flow. The small model answers everything first. The judge escalates only the hard requests. Escalated requests pay both models.

How to read this diagram

1. Every request enters the small model first and pays its cost. There is no way to skip it.
2. The judge scores the answer's confidence. Confident answers return directly.
3. Unsure answers escalate to the big model, then return.
4. What it is: a two-tier routing pattern. Why it exists: most requests are easy, and the small model costs a fifth of the big one. What breaks: a weak judge (escalates everything or nothing), or an escalation rate that drifts up until the cost math collapses.

Step 3: the economics that decide whether it is worth it

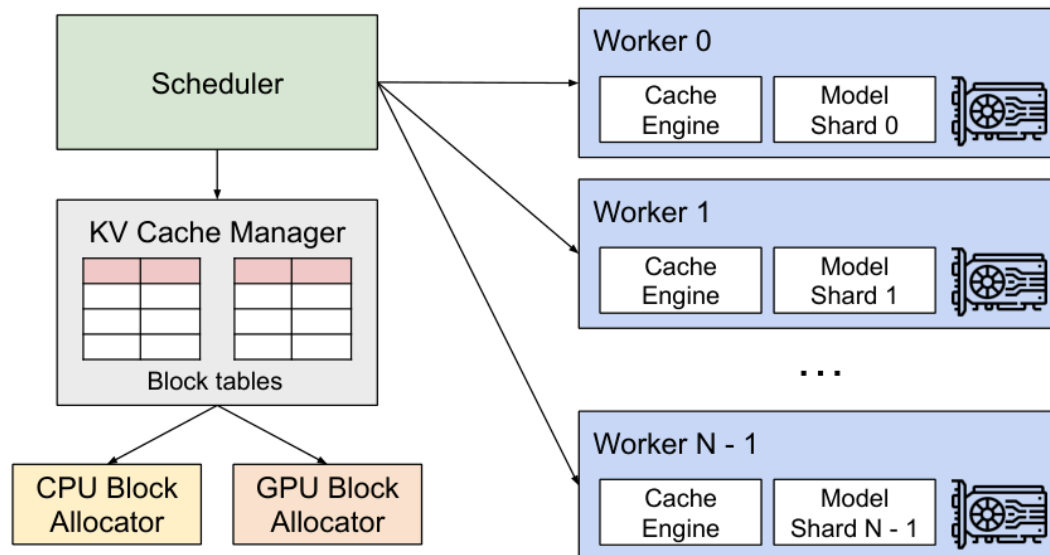
The figure's teaching point stands: the gate is the load-bearing piece. A sloppy gate either escalates too much (kills the savings) or accepts bad answers (kills quality while the token-cost metric looks great).

1. **Cascade when most traffic is easy.** 80/20 splits routinely cut cost 2-3x. Below ~60% easy, the margin thins out.
2. **Budget the escalation rate like money.** Track it on a dashboard. If it drifts from 20% to 50%, the cascade costs \$0.78 per 1M instead of \$0.44 and the p50 latency story breaks (past 50% escalation, even the median request hits the big model).
3. **Keep the big model warm.** A cold big model turns the escalation path into a latency disaster.
4. **Measure quality on accepted answers, not just the escalation rate.** A gate that silently accepts bad answers makes cost-per-token fall while cost-per-*successful*-outcome rises.

Decisions 1.1 through 1.5 are engine-agnostic. This one picks the engine that implements them, by traffic shape and fleet, not by brand.

1.6 Engine selection

All four engines do continuous batching now, so brand is not the decision. The ladder: pick by traffic shape and fleet, then tune the three scheduler details that decide real performance.



A central scheduler. A KV Cache Manager with per-sequence block tables. CPU/GPU block allocators. Workers run a cache engine plus a model shard each. The engine-selection rules in this section choose between these architectures. Figure: vLLM paper (Kwon et al., SOSP 2023).

Pick	When
vLLM	Broadest ecosystem and fastest iteration. Python, pip install, broad model coverage. The default starting point.
TensorRT-LLM	NVIDIA-only fleet and maximum perf-per-dollar, and you accept a per-model/GPU engine build step (slower iteration, faster runtime). Earliest FP8/FP4 kernel support.
SGLang	Agentic or multi-turn traffic with shared prefixes. RadixAttention shares prefixes across requests as a radix tree, and cache-aware scheduling routes to the worker already holding the prefix. Also the early mover on PD disaggregation.
TGI	Standard Hugging Face models with minimal fuss. Production-hardened defaults, less research velocity.

Three scheduler details decide real performance in any engine: **chunked prefill** (protects TPOT), **preemption policy** (evict the newest sequence, recompute later; a rising preemption rate is the distress signal), and **prefix-cache-aware routing** (skips redundant prefills for shared system prompts).

One last decision for the multi-tenant case: how to serve many fine-tuned variants without deploying one full copy per tenant.

1.7 Multi-tenant LoRA decision

When many fine-tuned variants share one base model, do not deploy one copy per tenant. The ladder:

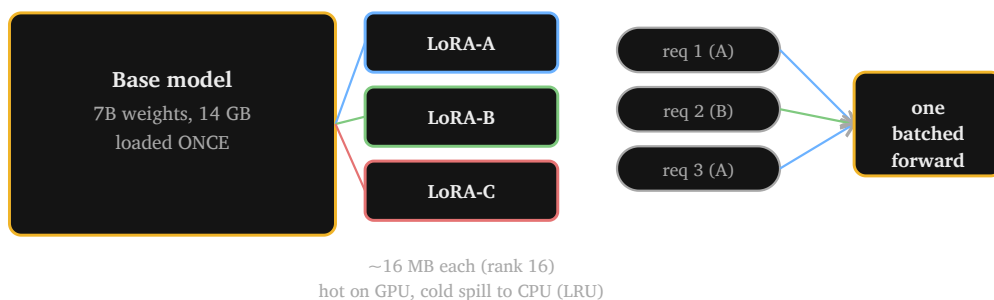
Step 1: load the base once, serve adapters on top

A rank-16 adapter on a 7B model is ~16 MB against 14 GB for the base. (Rank is the adapter's size knob: smaller rank, smaller adapter.) One GPU holds the base plus hundreds of tenants' adapters.

Step 2: batch tenants heterogeneously

Requests name their adapter and batch together in one forward pass, each request using its own adapter's weights. The figure below shows it.

Step 3: the three ops rules



Rule: heterogeneous adapters batch together. Warm adapters before traffic; a cold adapter pays a CPU fetch as a latency

How to read: the base model loads once (14 GB). Each tenant's adapter is a 16 MB chip clipped on. Requests naming different adapters still share one forward pass. Hot adapters stay on GPU. Cold ones spill to CPU under LRU (least recently used: the engine evicts the adapter idle longest first). A cold adapter hit is a CPU fetch, which is why the pack says to warm adapters before traffic.

1. Size `--max-loras` from the tenant traffic distribution: hot adapters stay on GPU, the rest spill to CPU RAM under LRU.
2. Warm adapters before production traffic: a cold adapter pays a CPU fetch as a latency spike.
3. Keep the runtime-load endpoint (`VLLM_ALLOW_RUNTIME_LORA_UPDATING`) trusted-admin-only, since it loads weight files from a request-supplied path.

Next: Section 2, the numbers behind every rule: roofline arithmetic, KV-cache bytes, latency targets, and dollars per million tokens.

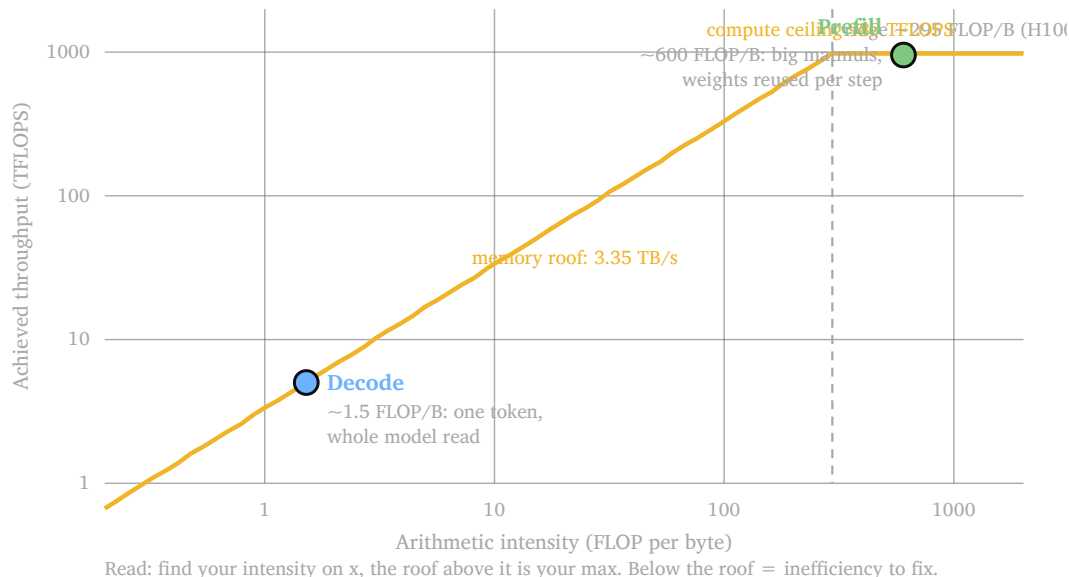
2. Must-memorize numbers and formulas

The rules tell you what to pick. They do not give you the arithmetic to size the pick.

Section 2 is the calculator behind Section 1: roofline math, KV-cache bytes per token, TTFT/TPOT targets, and cost per million tokens. It answers the question the rules leave open: how much will it cost, and will it fit?

2.1 Roofline arithmetic

The roofline says attainable performance is the minimum of two ceilings: what memory bandwidth allows and what compute allows. **Arithmetic intensity** = FLOPs / bytes moved. Below the **ridge point**, you are memory-bound. Above it, compute-bound.



How to read: find your arithmetic intensity on the x-axis; the gold roof above it is your ceiling. Decode sits at about 1.5 FLOP/byte on the memory slope (each token reads the whole model). Prefill sits at about 600 FLOP/byte under the compute ceiling (big matmuls reuse weights). Anything below the roof is inefficiency you can fix.

Roofline: bytes rule below the ridge, FLOPS rule above it

Schematic. Ridge point for H100 BF16: 295 FLOP/byte.

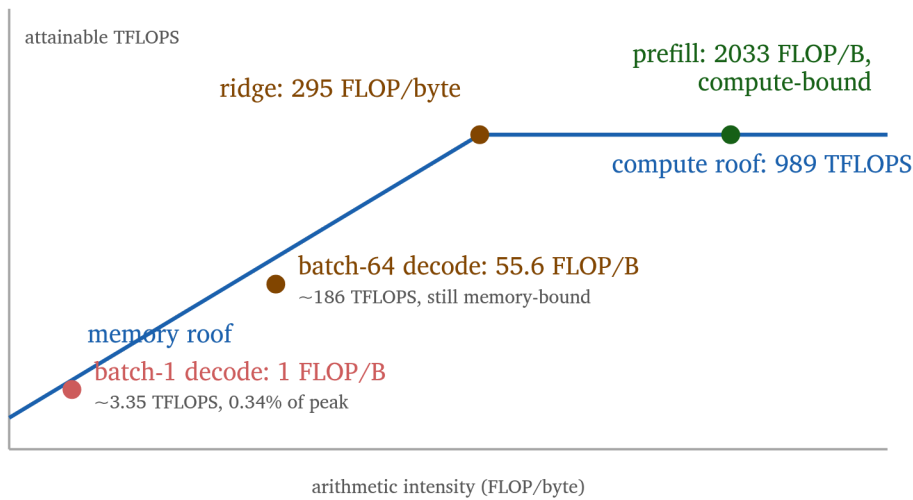
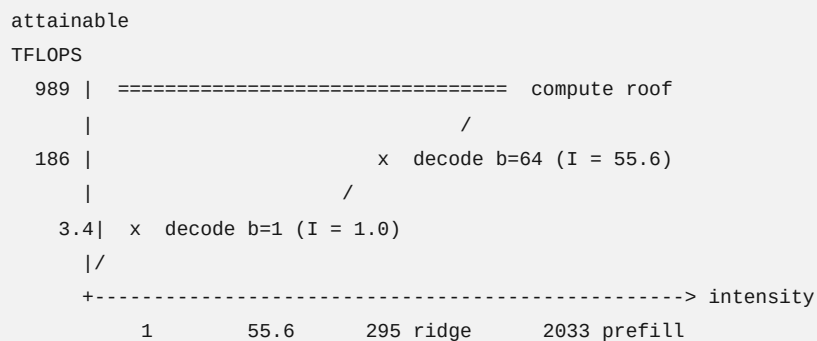


Figure 2.1. Roofline with this pack's operating points (schematic). Batch-1 decode sits far below the ridge at 0.34% of peak. Batch-64 decode climbs the memory roof. Prefill sits on the compute roof.

- Ridge point = peak FLOPS / peak bandwidth.
- H100 BF16 ridge: 989 TFLOPS / 3.35 TB/s = **295 FLOP/byte**.
- H100 FP8 ridge: 1979 / 3.35 = **591 FLOP/byte**.

$$\text{attainable} = \min(\text{bandwidth} \times \text{intensity}, \text{peak FLOPS})$$

Where: bandwidth is the memory bandwidth (3.35 TB/s on H100), intensity is arithmetic intensity in FLOP/byte, peak FLOPS is the compute ceiling (989 TFLOPS BF16 on H100). Below the ridge point the first term wins (memory-bound); above it, the second (compute-bound).



How to read this diagram

1. The flat top is the compute roof: 989 TFLOPS BF16, unreachable for decode. The rising slope is the memory roof: bandwidth x intensity.
2. Decode at batch 1 sits at intensity 1, deep on the slope: 3.35 TFLOPS attainable, 0.34% of peak.
3. Batch 64 slides right to intensity 55.6: 186 TFLOPS, still memory-bound, 55x the throughput at nearly the same step time.
4. Prefill of 2k tokens sits at intensity ~ 2033 , past the ridge, pinned to the compute roof.
5. Teaching point: batching moves decode right along the slope. Nothing about "faster math" matters until you cross the ridge.

The three worked cases (70B BF16 on H100), verified by the classifier below:

Case	FLOPs	Bytes	Intensity	Ceiling	Step time
Decode, batch 1	140 GFLOP	140 GB	1.0	3.35 TFLOPS (0.34% of peak)	41.8 ms -> ~ 24 tok/s
Decode, batch 64, avg 1k ctx	8.96 TFLOP	161 GB	55.6	186 TFLOPS (18.9% of peak)	48 ms for 64 tok -> $\sim 1,333$ tok/s
Prefill, 2k tokens, batch 1	2.87e14	141 GB	~ 2033	989 TFLOPS (compute-bound)	0.29 s

Per token, prefill is roughly **300x cheaper** than decode: it reuses each weight read across 2048 tokens. The GPU at batch-1 decode is a \$30,000 space heater running at one-third of one percent of its compute.

```
# roofline.py - place a serving phase on the H100 roofline.
# WHAT: from FLOPs and bytes, compute intensity, the binding roof,
# and the attainable ceiling. This is the whole roofline in one function.
# WHY: it tells you whether to buy bandwidth (batch, quantize) or
# FLOPs (nothing helps decode; prefill is already compute-bound).
# WHAT BREAKS: passing bytes as element counts (forget the x2 for BF16).
# Intensity inflates and a memory-bound phase looks compute-bound.
# Units are the diagnosis: FLOPs per BYTE.

PEAK_BW = 3.35e12    # H100 HBM bandwidth, bytes/second
PEAK_BF16 = 989e12  # H100 dense BF16 tensor-core FLOPs

def diagnose(flops, nbytes, label):
    # Intensity: the x-axis position on the roofline chart.
    intensity = flops / nbytes
    # Ridge point: where the memory roof meets the compute roof.
    ridge = PEAK_BF16 / PEAK_BW
    # Attainable = the lower of the two ceilings.
    ceiling = min(PEAK_BW * intensity, PEAK_BF16)
    side = "memory-bound" if intensity < ridge else "compute-bound"
    print(f"{label:28s} I={intensity:7.1f} {side:12s} "
          f"ceil={ceiling/1e12:6.1f} TFLOPS ({ceiling/PEAK_BF16:.2%} of peak)")
    return ceiling

d1 = diagnose(140e9, 140e9, "decode b=1")
d64 = diagnose(64 * 140e9, 161e9, "decode b=64")
p = diagnose(2 * 70e9 * 2048, 141e9, "prefill 2k tok")
print(f"decode b=1 step:  {140e9/d1*1000:.1f} ms -> {d1/140e9:.0f} tok/s")
print(f"decode b=64 step: {64*140e9/d64*1000:.1f} ms -> {d64/140e9:.0f} tok/s")
print(f"prefill 2k time:  {2.87e14/p*1000:.0f} ms")
```

```
decode b=1                I=    1.0 memory-bound ceil=    3.4 TFLOPS (0.34% of
peak)
decode b=64               I=   55.7 memory-bound ceil=  186.4 TFLOPS (18.85%
of peak)
prefill 2k tok            I= 2033.5 compute-bound ceil= 989.0 TFLOPS (100.00%
of peak)
decode b=1 step:  41.8 ms -> 24 tok/s
decode b=64 step: 48.1 ms -> 1332 tok/s
prefill 2k time:  290 ms
```

The roofline says what is attainable per byte moved. The KV cache decides how many bytes each token costs you. This is the capacity equation of serving.

2.2 KV-cache bytes per token

The KV cache stores, per layer, the keys and values of every processed token. Its size caps concurrent sequences, which caps batch size, which caps throughput. **This is the capacity equation of serving.**

$$\text{KV bytes per token} = 2 \times \text{layers} \times \text{KV-heads} \times \text{head-dim} \times \text{bytes-per-value}$$

Where: the 2 counts keys and values. Layers is the transformer depth. KV-heads is the number of key/value heads (fewer with grouped-query attention). Head-dim is the per-head dimension. Bytes-per-value is the dtype size (2 for BF16).

Worked: 70B-class (80 layers, 8 KV heads, head dim 128, BF16): $2 \times 80 \times 8 \times 128 \times 2 = 327,680 \text{ bytes} = 320 \text{ KiB}$ per token.

For a 70B-class model (80 layers, 8 KV heads via grouped-query attention, head dim 128, BF16). Grouped-query attention shares each KV head across a group of query heads, so fewer KV heads means a smaller cache. $2 \times 80 \times 8 \times 128 \times 2 = 327,680 \text{ bytes} = 320 \text{ KiB}$ per token.

Context	KV per sequence (BF16)	What it means
1k tokens	320 MB	trivial
8k tokens	2.56 GB	noticeable
32k tokens	10.2 GB	a real budget line
128k tokens	41 GB	more than half an 80 GB GPU

Two consequences that drive real decisions:

- **Long context is a batch-size tax.** At 32k context each sequence costs ~10 GB. A GPU with 10 GB of KV headroom fits exactly one such sequence. Throughput collapses, not because the GPU is slow, but because there is no room for a batch.
- **GQA is a serving feature.** Cutting KV heads from 64 to 8 cut the cache 8x. A 70B model with GQA can serve cheaper than a 40B model with full multi-head attention at long context. When comparing models for serving, KV bytes per token matters as much as parameter count.

The most common capacity mistake: sizing for weights and forgetting the cache. A 70B model in BF16 needs 140 GB for weights (two 80 GB GPUs with tensor parallelism). With 10 GB of KV headroom per GPU you serve roughly three concurrent 8k sequences per GPU (verified below). If traffic needs thirty, you need more GPUs or quantization. No kernel tuning changes that.

```
# kv_capacity.py - the capacity equation as a calculator.
# WHAT: per-token KV bytes from the architecture, then per-sequence
# cost and max concurrent sequences from a headroom budget.
# WHY: this is the arithmetic behind every OOM and every batch-size
# limit. Run it before choosing max-sequences, not after the page.
# WHAT BREAKS: forgetting the factor of 2 (keys AND values) halves
# the estimate and schedules an OOM at config time.

def kv_bytes_per_token(layers, kv_heads, head_dim, bytes_per=2):
    # 2 = one key vector and one value vector per layer per token.
    return 2 * layers * kv_heads * head_dim * bytes_per

BPT = kv_bytes_per_token(80, 8, 128, 2) # 70B-class GQA, BF16
print(f"per-token KV bytes: {BPT:,} = {BPT/1024:.0f} KiB/token")

for ctx in (1024, 8192, 32768, 131072):
    print(f"  ctx {ctx:>6,}: {BPT*ctx/2**30:5.2f} GiB per sequence")

for ctx, headroom_gib in [(8192, 10), (32768, 10), (8192, 40)]:
    n = int(headroom_gib * 2**30 // (BPT * ctx))
    print(f"  ctx {ctx}, {headroom_gib} GiB headroom -> {n} sequences max")

# GQA vs full multi-head attention, same model size:
print(f"full-MHA per-token: {kv_bytes_per_token(80,64,128,2)/1024:.0f} KiB "
      f"(8x the GQA figure: architecture is a serving decision)")
```

```
per-token KV bytes: 327,680 = 320 KiB/token
  ctx 1,024: 0.31 GiB per sequence
  ctx 8,192: 2.50 GiB per sequence
  ctx 32,768: 10.00 GiB per sequence
  ctx 131,072: 40.00 GiB per sequence
  ctx 8192, 10 GiB headroom -> 4 sequences max
  ctx 32768, 10 GiB headroom -> 1 sequences max
  ctx 8192, 40 GiB headroom -> 16 sequences max
```

Size capacity first. Now latency: what the user feels, and the two numbers that define it.

KV-cache bytes per token: the capacity equation of serving

KV bytes per token = 2 x layers x KV-heads x head-dim x bytes-per-value

the 2 counts keys and values; KV-heads is fewer with grouped-query attention

worked, 70B-class: $2 \times 80 \times 8 \times 128 \times 2 = 327,680$ bytes = 320 KiB per token

fewer KV heads means a smaller cache: GQA shares each KV head across query heads

1k tokens	320 MB	trivial
8k tokens	2.56 GB	noticeable
32k tokens	10.2 GB	a real budget line
128k tokens	41 GB	more than half an 80 GB GPU

Long context is a batch-size tax: at 32k each sequence costs ~10 GB, so 10 GB of KV headroom fits exactly one.

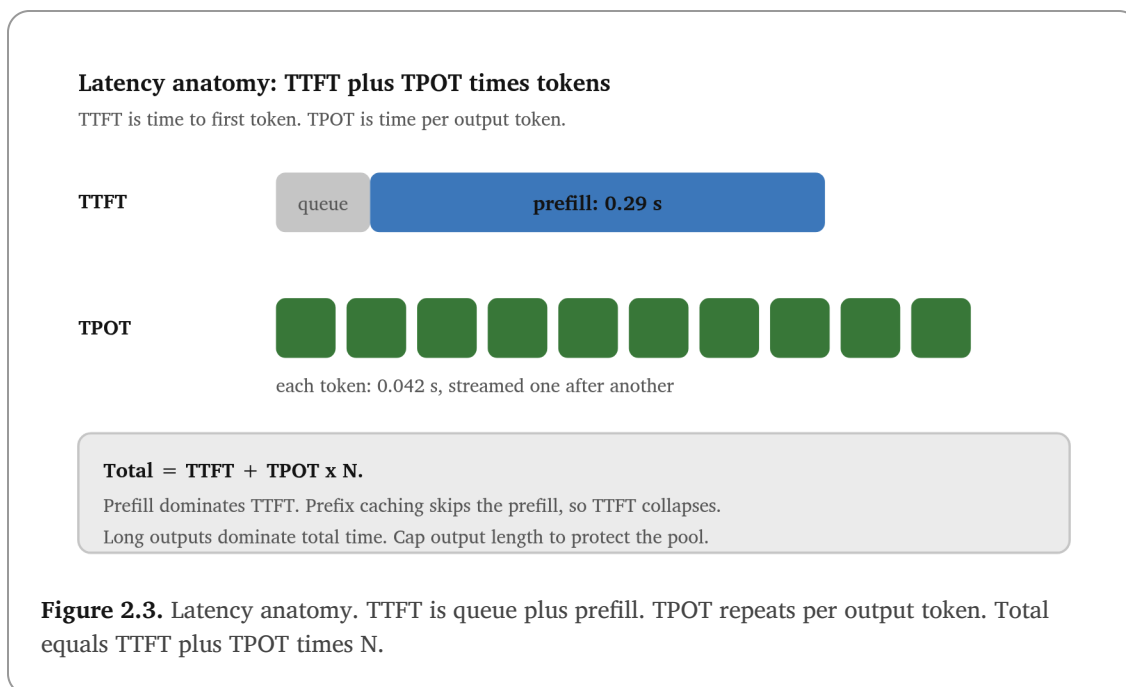
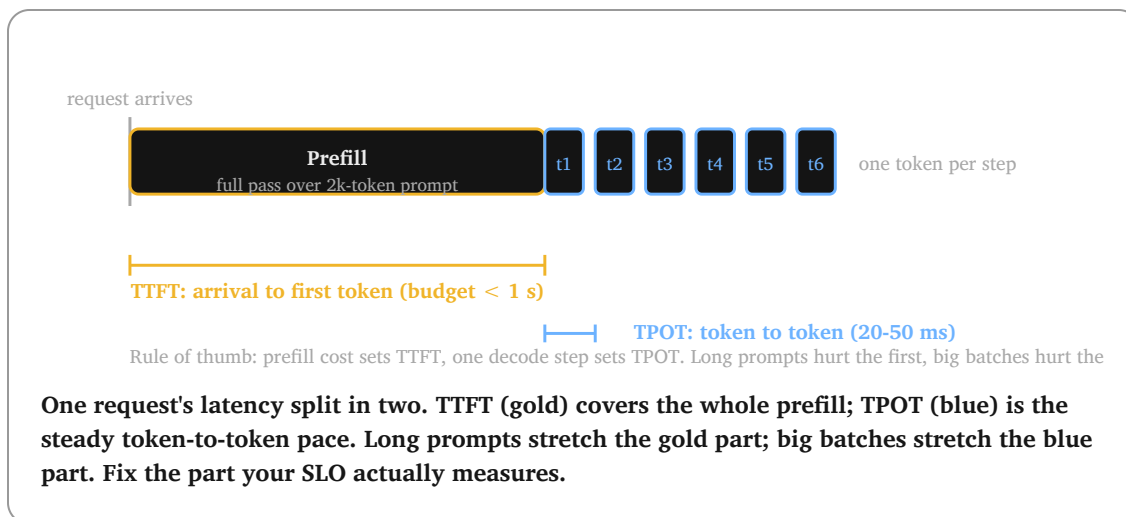
Figure 2.2. KV-cache bytes per token: the capacity equation of serving. Worked for the 70B-class model: $2 \times 80 \times 8 \times 128 \times 2 = 320$ KiB per token. At 32k context each sequence costs ~10 GB, so 10 GB of KV headroom fits exactly one sequence.

How to read it.

1. Read the formula top to bottom: 2 for keys and values, times layers, times KV heads, times head dim, times bytes per value.
2. The worked row plugs in the 70B-class numbers (80 layers, 8 KV heads via grouped-query attention, head dim 128, BF16) and lands on 320 KiB per token.
3. The table scales it: 1k tokens is 320 MB (trivial), 32k is 10.2 GB (a real budget line), 128k is 41 GB (more than half an 80 GB GPU).
4. The punchline: long context is a batch-size tax. KV size caps concurrent sequences, which caps batch size, which caps throughput.

2.3 TTFT and TPOT targets

Split latency into two parts, because they have different causes and different fixes:



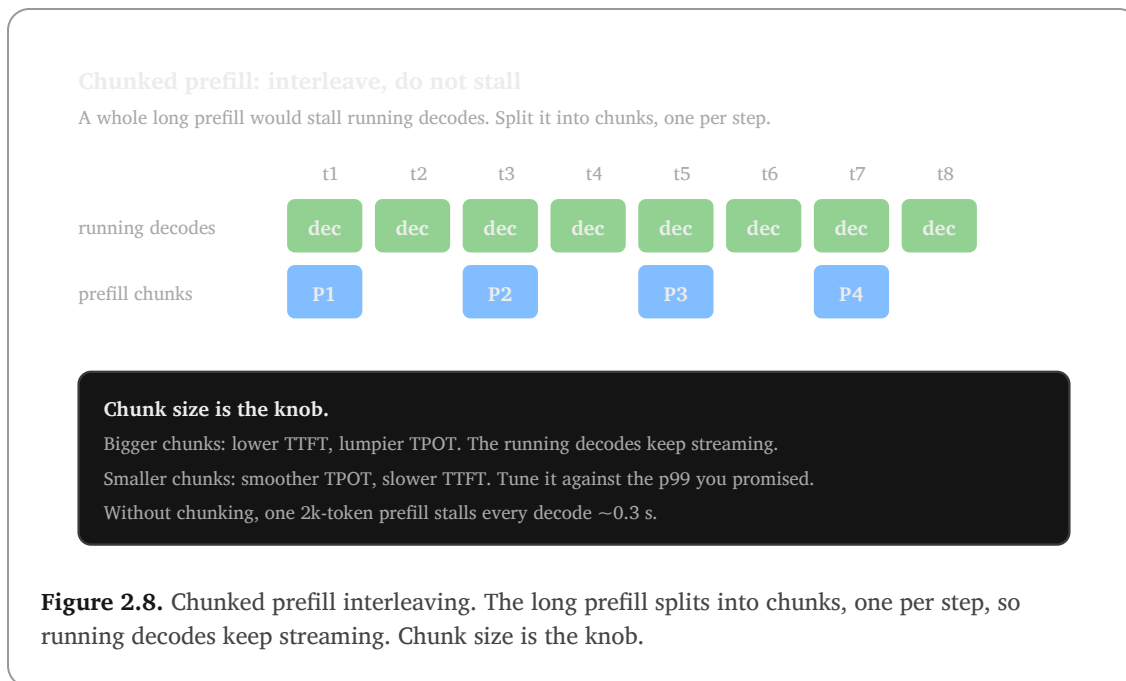
- **TTFT (time to first token)** = queue wait + prefill time. Dominated by prefill for long prompts, by queueing under load. This is responsiveness.
- **TPOT (time per output token)** = average decode step time. This is the streaming reading speed.

Reference points (illustrative, from the track's worked setups):

- Chat TTFT budgets live in the hundreds of milliseconds. A **tight TTFT SLO is under ~1 s**.
- Cascade fast path: small model TTFT ~150 ms. Escalated path: ~150 ms (small) + ~50 ms (gate + routing) + ~400 ms (big) = ~600 ms.
- TPOT stays flat as batch grows, until memory bandwidth saturates. A TPOT that climbs with batch is the signal you crossed saturation.

- Report **p99** and **p999**, not the mean. The mean hides the 1% of users having a terrible experience, and p99 can look fine while p999 is 10x worse. One stuck request in a batch can poison the whole batch's latency.

Latency targets are set. Now the price tag: the unit-cost formula that turns a fleet into dollars per million tokens.



2.4 Dollars per million tokens

The unit-cost formula:

$$\$/1\text{M tokens} = \text{GPU price} \times \text{GPUs} / (\text{tokens/sec} \times 3600 \times \text{utilization}) \times 1,000,000$$

Where: GPU price is dollars per GPU-hour. GPUs is the fleet size. Tokens/sec means sustained throughput. Utilization is the fraction of time the fleet does useful work (the silent multiplier). The 1,000,000 converts per-token to per-million.

Worked on 2 H100s at \$3/hr, ~1,500 tok/s sustained:

Utilization	\$/1M tokens
100%	\$1.11
80%	\$1.39
60%	\$1.85
50%	\$2.22

Utilization is the silent multiplier. A 30-point utilization gain (50% to 80%) cuts unit cost 37% with zero kernel changes. That is why capacity planning sits beside batching and quantization as a first-class lever.

The batching ladder (same fleet). Batch 1 at ~120 tok/s costs **\$13.89/1M**. Batch 8 at ~700 tok/s costs **\$2.38/1M**. Batch 64 at ~1,500 tok/s costs **\$1.11/1M**. Same GPUs, 12.5x cost difference.

Then the two honesty multipliers. **Cost per successful outcome** = raw cost per request / (utilization x success rate). For a 2,000-token request: raw \$0.0022; at 15% utilization \$0.015; at 70% success rate **\$0.021 per good answer**. The raw number said two-tenths of a cent. The honest number says two cents.

raw GPU cost per request:	\$0.0022	#
at 15% utilization:	\$0.0147	#####
at 70% success rate (honest):	\$0.0212	#####

How to read this diagram

1. The first bar is the raw GPU cost for one 2,000-token request on a fully loaded 70B fleet: about \$0.002.
2. The second bar adds the utilization layer. Idle GPUs still bill, so at 15% utilization the same request costs ~\$0.015.
3. The third bar adds the success layer. Retries and fallbacks mean each good answer carries $1/0.70 = 1.43$ attempts: ~\$0.021.
4. Teaching point: the honest metric is the third bar, and utilization is usually the biggest lever, bigger than the GPU price.

```

# serving_cost.py - unit cost, honest cost, cascade cost, spot math.
# WHAT: four small functions that price a serving design end to end.
# WHY: every fleet argument is really an argument about one of these
# numbers. Writing them down ends the argument.
# WHAT BREAKS: dropping the utilization term (assumes 100% util,
# understates real cost 3-10x) or the success-rate term (a "cheaper"
# model that hallucinates more can cost more per good answer).

def cost_per_million_tokens(gpu_price_per_hr, num_gpus,
                           tokens_per_sec, utilization=1.0):
    # Fleet bill per hour divided by tokens actually produced that hour.
    fleet_cost_per_hr = gpu_price_per_hr * num_gpus
    tokens_per_hr = tokens_per_sec * 3600 * utilization
    return fleet_cost_per_hr / tokens_per_hr * 1_000_000

def cost_per_successful_outcome(cost_per_request, utilization, success_rate):
    # Idle time and failed attempts are real money. Divide them in.
    return cost_per_request / (utilization * success_rate)

def cascade_cost(small_per_1m, big_per_1m, escalation_rate, gate_per_1m=0.0):
    # Every request pays small + gate. Escalated ones pay big too.
    # Forgetting the small-model cost on escalated requests flatters
    # the math: the escalation tax is small + gate + big.
    return small_per_1m + gate_per_1m + escalation_rate * big_per_1m

# 70B on 2x H100 at $3/hr, 1500 tok/s sustained.
for util in (1.0, 0.60, 0.50, 0.80):
    print(f"util {util:4.0%} -> "
          f"${cost_per_million_tokens(3.0, 2, 1500, utilization=util):.2f}/1M")
# 2000-token request: raw vs honest.
raw = cost_per_million_tokens(3.0, 2, 1500) / 1e6 * 2000
print(f"2000-tok request: raw ${raw:.4f} -> honest "
      f"${cost_per_successful_outcome(raw, 0.15, 0.70):.4f} per good answer")
# Cascade: 8B at $0.22/1M, 70B at $1.11/1M, 80/20 split, $0.05 judge gate.
print(f"cascade 80/20:      ${cascade_cost(0.22, 1.11, 0.20):.2f}/1M "
      f"({1.11/cascade_cost(0.22, 1.11, 0.20):.1f}x cheaper than big-only)")
print(f"cascade 80/20 + judge: ${cascade_cost(0.22, 1.11, 0.20, 0.05):.2f}/1M")
# Spot: 65% cheaper, ~5% of spot-served requests retried on reserved.
print(f"spot effective discount: {1 - (0.35 + 0.35*0.05):.1%} "
      f"(65% off, 5% retry cost on the 35% you still pay)")

```

```

util 100% -> $1.11/1M
util 60% -> $1.85/1M
util 50% -> $2.22/1M
util 80% -> $1.39/1M
2000-tok request: raw $0.0022 -> honest $0.0212 per good answer
cascade 80/20:      $0.44/1M (2.5x cheaper than big-only)
cascade 80/20 + judge: $0.49/1M
spot effective discount: 63.2% (65% off, 5% retry cost on the 35% you still
pay)

```

Cost per token has a price. Utilization and MFU are the levers that move it. The discipline: read them together.

Dollars per million tokens: the batching ladder and the honesty multipliers

The batching ladder: same fleet, 12.5x cost difference

batch 1	~120 tok/s	\$13.89 /1M
batch 8	~700 tok/s	\$2.38 /1M
batch 64	~1,500 tok/s	\$1.11 /1M

The honesty multipliers: cost per successful outcome = raw cost / (utilization x success rate)

raw GPU cost per request: \$0.0022

at 15% utilization: \$0.0147

at 70% success rate (honest): \$0.0212 per good answer

The raw number said two-tenths of a cent. The honest number says two cents.

Figure 2.4. Dollars per million tokens: the batching ladder and the honesty multipliers. Same fleet, 12.5x cost difference from batch 1 to batch 64. And the honest cost per good answer is ten times the raw GPU cost per request.

How to read it.

1. The ladder: batch 1 at ~120 tok/s costs \$13.89 per million tokens. Batch 64 at ~1,500 tok/s costs \$1.11. Batching is a first-class cost lever.
2. Three multipliers. Raw GPU cost per request is \$0.0022. At 15 percent utilization it is \$0.0147. At a 70 percent success rate the honest cost is \$0.0212 per good answer.
3. Utilization is the silent multiplier: a 30-point utilization gain (50 to 80 percent) cuts unit cost 37 percent with zero kernel changes.

2.5 Utilization and MFU

Two efficiency numbers, and the discipline to read them together:

- **Read utilization with intensity.** Section 2.1 worked the number: batch-1 decode reaches ~0.34% of an H100's BF16 peak. That is not a bug in your code; it is the physics of reading 140 GB to produce one token. Fix it with batching and quantization, not kernels.
- **MFU (model FLOPs utilization)** = achieved FLOPs / peak FLOPS. The roofline version: fraction of the *attainable ceiling*. Kernel A (FP32 vector add) hit ~8% of its ceiling: a memory-access-pattern suspect. Kernel B (FP8 GEMM) hit ~60%: compute-bound and healthy, look at precision and instruction mix if you want more. (GEMM is general matrix multiply, the core neural-net operation.)

- **Utilization without intensity is vanity.** SM utilization counts warps with work to do, not useful work done. (An SM, streaming multiprocessor, is one of the GPU's compute units; a warp is a group of 32 threads that run in lockstep.) A kernel doing wasted strided reads can show 90% SM utilization while moving a tenth of the bytes it should. Intensity tells you what the work is worth; utilization tells you the SM was awake. You need both.

$$\text{MFU} = \text{achieved FLOPs} / \text{peak FLOPs}$$

Where: achieved FLOPs is the model math the run actually sustains (counts only useful math: communication, recompute, and bubbles lower it), peak FLOPs is the hardware ceiling at the precision used. The roofline version: the fraction of the *attainable* ceiling.

Healthy: 40 to 60 percent on a large run; below 30 percent, fix systems before science.

Worked: Kernel B (FP8 GEMM) hit ~60% of its ceiling: $0.60 \times 1,979 \text{ TFLOPs (H100 FP8 peak)} = 1,187 \text{ TFLOPs achieved}$. Compute-bound and healthy.

Quantization was a decision in 1.2. Here is what each method costs in quality, honestly.

Utilization and MFU: read them together

Utilization without intensity is vanity

SM utilization counts warps with work to do, not useful work done.

A kernel doing wasted strided reads can show 90% SM utilization while moving a tenth of the bytes it should.

MFU = achieved FLOPs / peak FLOPs

the roofline version: fraction of the attainable ceiling.

Kernel A (FP32 vector add) hit ~8% of its ceiling: a memory-access-pattern suspect.

Kernel B (FP8 GEMM) hit ~60%: compute-bound and healthy.

Batch-1 decode reaches ~0.34% of an H100's BF16 peak. That is not a bug in your code:

it is the physics of reading 140 GB to produce one token.

Intensity tells you what the work is worth; utilization tells you the SM was awake. You need both.

Figure 2.5. Utilization and MFU: read them together. SM utilization counts warps that are awake, not useful work done. MFU means achieved FLOPs over peak FLOPs: the roofline fraction of the attainable ceiling.

2.6 Quantization accuracy deltas

What each method costs in quality, honestly:

- **FP8 W8A8 on Hopper:** near-zero accuracy loss on most models. The default serious quantization, not an exotic option. vLLM reports up to 1.6x throughput.

- **The perplexity trap, quantified** (the rule is in 1.2): a 4-bit model can show unchanged perplexity with a 6-point drop on math tasks. The cause is almost always calibration (wrong data, too few samples, per-tensor scales on outlier layers), not the bit-width.
- **int8 KV cache:** nearly lossless in production engines. **int4 KV:** increasingly available with minor trade-offs. **2-bit (KIVI-style):** the research frontier. (KIVI is a 2-bit KV-cache quantization method; its trick is asymmetry, covered below.)
- **FP8 formats:** E4M3 = 1 sign + 4 exponent + 3 mantissa bits, range +/-448, used for weights and activations (precision). E5M2 = 1 + 5 + 2, range +/-57,344, used where range matters. E4M3 has no infinities: overflow saturates, which silently biases sums. E5M2 has infinities: overflow becomes inf and poisons everything downstream.
- **Block-wise microscaling (the decision that matters more than the format):** one scale per block of 128 values quarantines outliers. Per-tensor scaling with one loud outlier erases ~100% of normal values to zero; block-128 erases ~3%; block-32 ~0.8% (verified below). DeepSeek-V3 training used 128 and 128x128 blocks with FP32 scales, all E4M3, loss error under 0.25% vs BF16. Inference formats continue the pattern: MXFP8 (32 values, one E8M0 power-of-two scale, 8.25 bits/element), NVFP4 on Blackwell (4-bit, blocks of 16).

```

# fp8_blocks.py - why block-wise scaling beats per-tensor scaling.
# WHAT: quantize a tensor with one global scale vs one scale per block
# of 128 (then 32), and count values erased to zero.
# WHY: one outlier's oversized scale zeroes every normal value that
# shares it. Block-wise scaling confines the damage to the outlier's
# own block. The scale placement matters more than the format.
# WHAT BREAKS: block_size=1. Error drops to ~0, but you store a 4-byte
# scale per 1-byte value: 40 bits/element, worse than fp32. Block size
# trades accuracy for scale overhead; 128 and 32 are the compromises.

import numpy as np

E4M3_MAX = 448.0 # largest finite E4M3 value; codes saturate here

def quantize(x, block_size):
    # One scale per block: the block's max abs value maps to 448.
    n_blocks = int(np.ceil(len(x) / block_size))
    # Pad the tail so every block is full; padding is discarded after.
    padded = np.pad(x, (0, n_blocks * block_size - len(x)))
    blocks = padded.reshape(n_blocks, block_size)
    amax = np.abs(blocks).max(axis=1, keepdims=True)
    # All-zero blocks get scale 1.0: avoids division by zero, maps 0->0.
    scale = np.where(amax > 0, amax / E4M3_MAX, 1.0)
    codes = np.clip(np.round(blocks / scale), -E4M3_MAX, E4M3_MAX)
    return codes.reshape(-1)[:len(x)], scale.reshape(-1)

def erased_fraction(x, x_hat):
    # A value that rounds to zero is not noisy, it is gone. This is
    # the visible damage of a too-large scale.
    big = np.abs(x) > 0.05
    return float((x_hat[big] == 0).mean())

rng = np.random.default_rng(1)
x = (rng.standard_normal(4096) * 0.5).astype(np.float32)
x[1234] = 10000.0 # one loud outlier, like real activation spikes
for name, bs in [("per-tensor", len(x)), ("block-128", 128), ("block-32", 32)]:
    codes, scales = quantize(x, bs)
    nb = int(np.ceil(len(x) / bs))
    x_hat = (codes.reshape(nb, bs) * scales.reshape(nb, 1)).reshape(-1)
    [:len(x)]
    print(f"{name:10s}: {nb:4d} scales, erased {erased_fraction(x,
x_hat):.1%}")

```

```

per-tensor:    1 scales, erased 100.0%
block-128 :   32 scales, erased 3.2%
block-32  :  128 scales, erased 0.8%

```

The KIVI asymmetry for KV-cache quantization (keys per-channel, values per-token), verified:

```
# kivi.py - keys and values want different quantization layouts.
# WHAT: quantize a synthetic key matrix two ways (per-channel vs
# per-token) and measure the damage on the 63 quiet channels.
# WHY: keys carry persistent per-channel outliers (attention sinks: a few key
# channels whose values stay large across all tokens).
# Per-token scales stretch to cover the outlier channel, crushing
# every normal value in that token. Per-channel confines each
# outlier to its own scale.
# WHAT BREAKS: judging by max abs error. The outlier dominates that
# metric either way. The real damage is on the quiet channels, which
# carry most of the signal: per-token zeroes 41% of their values.
```

```
import numpy as np

def quantize_per_channel(x, bits=4):
    # One (min, max) pair per channel: reduce down the token axis.
    lo = x.min(axis=0, keepdims=True)
    hi = x.max(axis=0, keepdims=True)
    scale = np.maximum(hi - lo, 1e-12) / (2 ** bits - 1)
    return np.round((x - lo) / scale).astype(np.int32) * scale + lo

def quantize_per_token(x, bits=4):
    # One (min, max) pair per token: reduce across the channel axis.
    lo = x.min(axis=1, keepdims=True)
    hi = x.max(axis=1, keepdims=True)
    scale = np.maximum(hi - lo, 1e-12) / (2 ** bits - 1)
    return np.round((x - lo) / scale).astype(np.int32) * scale + lo

rng = np.random.default_rng(1)
keys = rng.normal(0, 0.1, size=(256, 64)).astype(np.float32)
keys[:, 5] = rng.normal(0, 3.0, size=256) # persistent outlier channel
qc = quantize_per_channel(keys)
qt = quantize_per_token(keys)
quiet = np.ones(64, dtype=bool)
quiet[5] = False
ec = np.abs(keys[:, quiet] - qc[:, quiet]).mean()
et = np.abs(keys[:, quiet] - qt[:, quiet]).mean()
big = np.abs(keys[:, quiet]) > 0.05
zc = float(((qc[:, quiet] == 0) & big).mean())
zt = float(((qt[:, quiet] == 0) & big).mean())
print(f"quiet-channel mean abs err, per-channel: {ec:.4f}")
print(f"quiet-channel mean abs err, per-token:  {et:.4f} ({et/ec:.1f}x
worse)")
print(f"quiet values erased to 0, per-channel: {zc:.1%}")
print(f"quiet values erased to 0, per-token:  {zt:.1%}")
```

```
quiet-channel mean abs err, per-channel: 0.0093
quiet-channel mean abs err, per-token:  0.0781 (8.4x worse)
quiet values erased to 0, per-channel: 0.0%
quiet values erased to 0, per-token:  41.0%
```

Two more KIVI details that make it work: the **residual window** (the most recent ~ 128 tokens stay in fp16 and quantize only as they age out; recent tokens dominate attention and keeping them exact is cheap) and streaming-friendly grouping (per-channel key scales need a full group before a scale exists, so the cache flushes in whole groups).

The KV cache sized the box. KV transfer sizes the link between the prefill box and the de-code box.

Quantization accuracy deltas: what each method costs

FP8 W8A8 on Hopper near-zero accuracy loss on most models
the default serious quantization; vLLM reports up to 1.6x throughput

int8 KV cache nearly lossless in production engines

int4 KV increasingly available, minor trade-offs

4-bit weights flat perplexity, 6-point drop on math tasks
the perplexity trap: cause is almost always calibration, not bit-width

2-bit (KIVI-style) the research frontier

FP8 formats: E4M3 (weights and activations, precision; overflow saturates)

vs E5M2 (range; overflow becomes inf and poisons everything downstream).

Block-128 microscaling quarantines outliers: per-tensor scaling erases $\sim 100\%$ of normal

values to zero with one loud outlier; block-128 erases $\sim 3\%$.

Figure 2.6. Quantization accuracy deltas: what each method costs, honestly. FP8 W8A8 on Hopper is the default serious quantization at near-zero loss. The 4-bit row is the perplexity trap: flat perplexity with a 6-point drop on math tasks, caused by calibration, not bit-width.

2.7 KV transfer math

Transfer bytes per request are exactly the prompt's KV cache: prompt tokens x 320 KiB/token for the 70B-class model.

Prompt	Transfer size	Over NVLink (~ 450 GB/s eff.)	Over IB NDR (~ 50 GB/s)
2k tokens	640 MB	~ 1.5 ms	~ 13 ms
8k tokens	2.56 GB	~ 6 ms	~ 54 ms
32k tokens	10.2 GB	~ 24 ms	~ 215 ms

Two conclusions: intra-node transfer is nearly free; cross-node is not. At 2k-8k prompts over NVLink, the transfer is single-digit milliseconds against a TTFT budget of hundreds of milliseconds. Over InfiniBand at 32k prompts, ~215 ms can blow a tight TTFT SLO by itself.

```
# transfer_gate.py - the disaggregation go/no-go check.
# WHAT: transfer milliseconds for a prompt over a link, then the two
# gates: under a quarter of the TTFT budget, AND cheaper than the
# prefill/decode interference it removes.
# WHY: disaggregation is a trade with a measured tax, not a universal
# win. These two comparisons are the trade, quantified.
# WHAT BREAKS: checking only the budget gate. A transfer that fits the
# budget but costs more than the interference it removes still loses.

BPT = 320 * 1024 # KV bytes per prompt token, 70B-class GQA

def transfer_ms(prompt_tokens, link_gbs):
    # link_gbs: effective GB/s. Result in milliseconds.
    return BPT * prompt_tokens / (link_gbs * 1e9) * 1000

for ctx, link, name in [(2048, 450.0, "NVLink"), (8192, 450.0, "NVLink"),
                        (32768, 450.0, "NVLink"), (32768, 50.0, "IB NDR")]:
    print(f"ctx {ctx:>5},: {name:6s} {transfer_ms(ctx, link):6.1f} ms")

# Gate: TTFT budget 800 ms, measured shared-pool interference 300 ms.
for ctx, link, name in [(2048, 450.0, "NVLink"), (8192, 450.0, "NVLink"),
                        (32768, 50.0, "IB NDR")]:
    t = transfer_ms(ctx, link)
    ok = t < 0.25 * 800 and t < 300
    print(f"gate ctx={ctx} {name}: transfer {t:.0f} ms -> "
          f"{'PASS' if ok else 'FAIL'}")
```

```
ctx 2,048: NVLink    1.5 ms
ctx 8,192: NVLink    6.0 ms
ctx 32,768: NVLink   23.9 ms
ctx 32,768: IB NDR  214.7 ms
gate ctx=2048 NVLink: transfer 1 ms -> PASS
gate ctx=8192 NVLink: transfer 6 ms -> PASS
gate ctx=32768 IB NDR: transfer 215 ms -> FAIL
```

One architecture change to the serving math: mixture of experts, where only a few experts run per token.

KV transfer math: the disaggregation go/no-go

Transfer bytes per request = prompt tokens x 320 KiB/token (70B-class)

		over NVLink (~450 GB/s)	over IB NDR (~50 GB/s)
2k tokens	640 MB	~1.5 ms	~13 ms
8k tokens	2.56 GB	~6 ms	~54 ms
32k tokens	10.2 GB	~24 ms	~215 ms

Intra-node transfer is nearly free; cross-node is not. At 32k over InfiniBand, ~215 ms can blow a tight TTFT SLO by itself.

The two gates: under a quarter of the TTFT budget, AND cheaper than the prefill/decode interference it removes.

Figure 2.7. KV transfer math: the disaggregation go/no-go. Transfer bytes per request are exactly the prompt's KV cache. Intra-node is nearly free; cross-node is not.

How to read it.

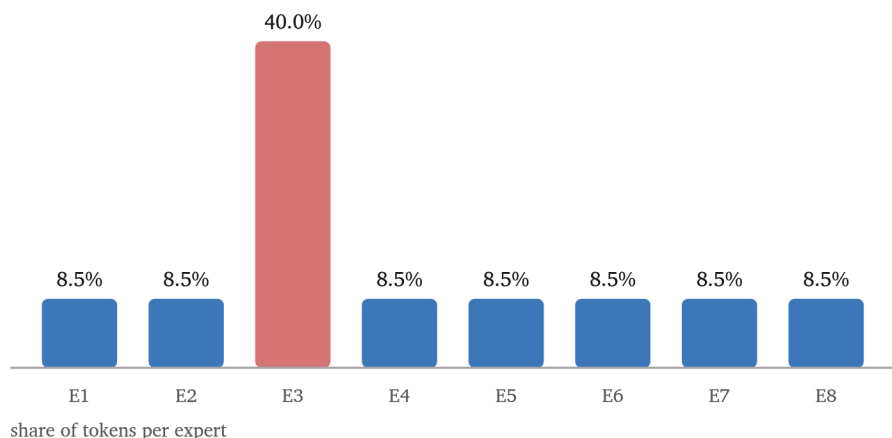
1. Find the prompt size in the left column, then read across: transfer size, time over NVLink (~450 GB/s), time over InfiniBand NDR (~50 GB/s).
2. At 2k-8k prompts over NVLink the transfer is single-digit milliseconds against a TTFT budget of hundreds of milliseconds: disaggregation wins big.
3. At 32k over InfiniBand, ~215 ms can blow a tight TTFT SLO by itself. The two gates: under a quarter of the TTFT budget, AND cheaper than the prefill/decode interference it removes.

2.8 MoE serving numbers

DeepSeek-V3, the canonical example: **671B total parameters, 256 routed experts plus one shared expert, only ~37B active per token** (8 routed + 1 shared). (Top-k routing: a small router scores every expert per token and sends the token to the k best; here 8, plus one always-on shared expert.) 94.5% of the feed-forward parameters sit idle for any given token.

One hot expert sets the pace for all eight

Routing is never uniform. The hottest rank sets the whole layer step time.



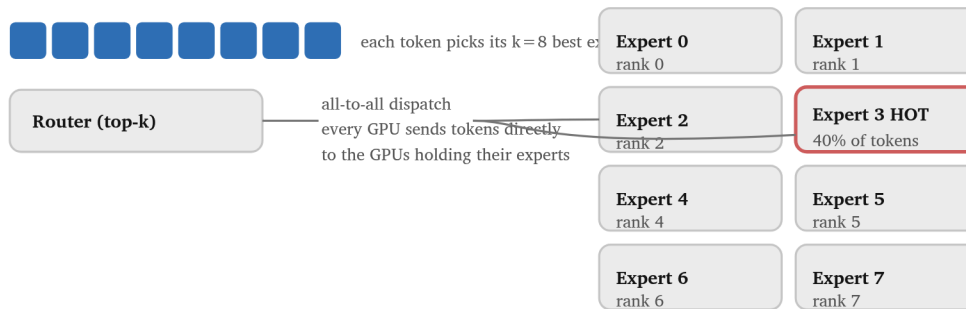
The fix is not faster experts. It is spreading the hot expert load. Alert when imbalance crosses $\sim 2x$.
Cost truth: only 37B of 671B parameters activate per token, but all experts must live on GPUs.

Figure 2.11. MoE expert imbalance. Expert 3 takes 40% of tokens while seven experts idle. The hottest rank sets the whole layer's step time.

- **Expert parallelism** shards experts across GPUs; tokens travel to their experts over all-to-all dispatch (every GPU sends tokens directly to every other GPU holding the needed experts). The network is part of the model now. In vLLM: `--enable-expert-parallel`.
- **Hot experts** are the failure mode. Routing is never uniform. One published measurement showed a single rank receiving the bulk of routed tokens because two hot experts lived on it, while other ranks idled. The hottest rank sets the layer's latency.
- **Monitor per-expert token counts.** Healthy is roughly uniform within $\pm 20\%$. One expert above **40% of tokens** is the alert threshold.
- **Fixes:** EPLB (expert parallel load balancer, `--eplb-config` in vLLM: redundant copies of hot experts on extra ranks), Waterfill in SGLang (rebalances experts across ranks: $+1.5\%$ to $+4.7\%$ throughput on DeepSeek-V3/R1-style workloads), LPLB in SGLang (a lighter-weight balancer: $+0.8\%$ to $+7.3\%$).
- **Cost truth:** top-k routing fixes active parameters per token (37B regardless of the 671B total). The cost of MoE is not compute. It is memory (all experts must live on GPUs), network (dispatch/combine), and the load-balance tax.

Tokens enter the MoE layer

671B total, but only ~37B active per token (8 routed + 1 shared)

**The hottest rank sets the layer's latency.**

Routing is never uniform. Here one rank does ~3.5x the average work, so the whole layer waits on it. Alert: one expert above 40% of tokens.

Fixes: EPLB (redundant copies of hot experts), Waterfill, LPLB.

MoE cost is not compute (37B active regardless of 671B total).

It is memory (all experts live on GPUs) + network (dispatch/combine) + load-balance tax.

Expert parallelism: the router sends each token to its 8 best experts over all-to-all dispatch, then combines the results. Only 37B of the 671B parameters activate per token, but all experts must live on GPUs. Routing is never uniform: the hottest rank (here Expert 3 with 40% of tokens) sets the whole layer's latency.

```
# moe_imbalance.py - how skewed is MoE routing?
# WHAT: divide the hottest expert's load by the average load.
# WHY: under expert parallelism the slowest rank sets the layer's
# latency. The average hides the straggler; the max IS the problem.
# WHAT BREAKS: averaging instead of taking the max. The whole point
# is the skew, and the mean erases it.

import random

def expert_imbalance(token_counts):
    # 1.0 = perfectly balanced. 3.5 = the hot expert does 3.5x the
    # average work, so the layer waits on it 3.5x longer.
    avg = sum(token_counts) / len(token_counts)
    return max(token_counts) / avg

rng = random.Random(7)
counts = [0] * 8
for _ in range(800):
    # Skewed routing: expert 3 attracts ~40% of tokens.
    expert = 3 if rng.random() < 0.40 else rng.choice([0, 1, 2, 4, 5, 6, 7])
    counts[expert] += 1
print("tokens per expert:", counts)
print(f"skewed imbalance factor: {expert_imbalance(counts):.2f}")
print(f"uniform imbalance factor: {expert_imbalance([100]*8):.2f}")
```

```
tokens per expert: [70, 71, 70, 353, 57, 57, 54, 68]
skewed imbalance factor: 3.53
uniform imbalance factor: 1.00
```

LoRA was a decision in 1.7. Here are its bytes: what one adapter really costs, and what a hundred cost naive versus shared.

2.9 Multi-tenant LoRA numbers

$$\text{adapter bytes} = 2 \times \text{layers} \times \text{matrices} \times \text{dim} \times \text{rank} \times \text{bytes-per-value}$$

Where: the 2 counts the A and B low-rank matrices. Layers is 32 for the 7B model.

Matrices is 2 (attention projections targeted). Dim is 4,096. Rank is 16. Bytes-per-value is 2 (BF16).

Worked: $2 \times 32 \times 2 \times 4,096 \times 16 \times 2 = 16,777,216$ bytes (~16 MB) per tenant, against 14 GB for the base model. One GPU holds the base plus hundreds of tenants' adapters.

Setup	GPU memory
Naive: 100 full model copies	1,400 GB (eighteen 80 GB GPUs before KV cache)
Shared base + 100 adapters	~15.6 GB (one GPU with room to spare)
Savings factor	~90x

The win is architectural, not a kernel optimization, and it stacks with the other levers in this pack.

All the numbers are in place. The fleet mix turns them into a buying plan: spot versus reserved, sized by the math above.

Multi-tenant LoRA: 16 MB per tenant against a 14 GB base

$$\text{adapter bytes} = 2 \times \text{layers} \times \text{matrices} \times \text{dim} \times \text{rank} \times \text{bytes-per-value}$$

worked: $2 \times 32 \times 2 \times 4,096 \times 16 \times 2 = \sim 16$ MB per tenant, against 14 GB for the base model
layers is 32 for the 7B model; the 2 counts the A and B low-rank matrices

naive: 100 full copies
1,400 GB
eighteen 80 GB GPUs before KV cache

shared base + 100 adapters
~15.6 GB
one GPU with room to spare

Savings factor ~90x. The win is architectural, not a kernel optimization.

Figure 2.9. Multi-tenant LoRA: 16 MB per tenant against a 14 GB base. 100 naive full copies need 1,400 GB (eighteen 80 GB GPUs); a shared base plus 100 adapters needs ~15.6 GB on one GPU. Savings factor ~90x, architectural, not a kernel optimization.

2.10 Fleet mix numbers

- Spot discount: **50-70% off on-demand**. Effective discount after realistic retry costs: **~60% +**.
- Size reserved for the **10th-percentile load**, not the mean. Peaks go on spot.
- Shadow rollout economics (50k requests/day). A shadow rollout mirrors production traffic to the new setup without serving from it, so you measure real behavior risk-free. A shadow week at \$0.014/request on 350k mirrored requests costs **\$4,900**. Promotion saves \$2,100/week forever. Breakeven **~2.3 weeks**. Budget the shadow fleet like a line item. At 100% mirroring you pay double for the whole shadow period.
- Promotion gates, written *before* the shadow starts. Quality within 0.5 points of baseline. p99 latency within 110% of baseline. Cost per request at or below baseline. Error-rate bump under 0.5 points. Ramp 1%, 10%, 50%, 100% with automatic rollback triggers. Test the rollback path before the ramp starts.

Next: Section 3, twelve ways serving stacks fail that look like mysteries until you have seen them once.

Fleet mix: spot discount, sizing rule, shadow economics

Spot discount: 50-70% off on-demand. Effective discount after realistic retry costs: ~60% +.

Size reserved for the 10th-percentile load, not the mean. Peaks go on spot.

shadow week: 350k mirrored requests x \$0.014 = \$4,900

promotion saves \$2,100/week forever; breakeven ~2.3 weeks

Promotion gates, written before the shadow starts:

- quality within 0.5 points of baseline
- p99 latency within 110% of baseline
- cost per request at or below baseline
- error-rate bump under 0.5 points

Ramp 1%, 10%, 50%, 100% with automatic rollback triggers. The rollback path is tested before the ramp starts.

Figure 2.10. Fleet mix: spot discount, sizing rule, shadow economics. Size reserved for the 10th-percentile load, not the mean; peaks go on spot. A shadow week at \$0.014/request on 350k mirrored requests costs \$4,900; promotion saves \$2,100/week forever; breakeven ~2.3 weeks.

3. Classic traps

The numbers are right and the fleet is still misbehaving. Section 3 covers the twelve failure modes that look like mysteries the first time. The twelve failure modes: coordinated omission, the KV OOM cliff, goodput versus throughput, and the preemption storm.

Twelve ways serving stacks fail that look like mysteries until you have seen them once.

The preemption storm feeds itself

More admitted sequences than KV blocks. The fix is admission control, not hardware.

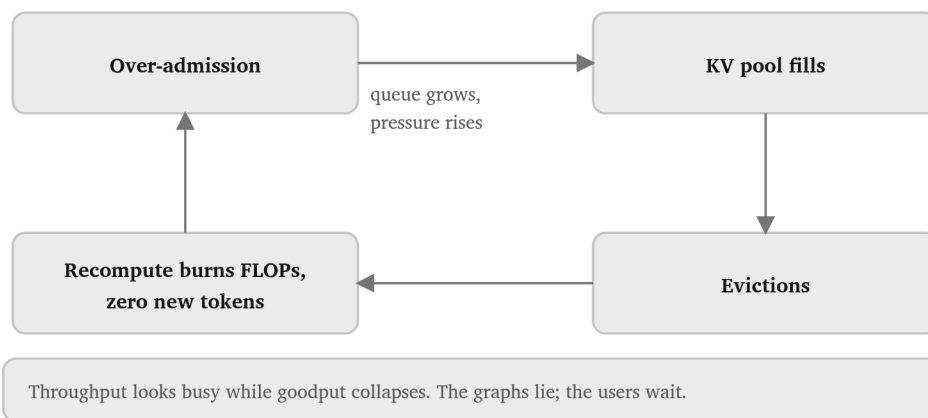


Figure 3.2. The preemption storm feedback loop. Over-admission fills the KV pool. Evictions force recompute. Recompute grows the queue. The queue raises pressure.

How to read this diagram

1. Over-admission starts the loop: more sequences enter than the KV pool can hold.
2. The pool fills. The scheduler evicts sequences to make room.
3. Evicted sequences recompute from scratch, burning FLOPs that produce zero new tokens.
4. Recompute keeps the queue long and pressure high, which causes more evictions.
5. What it is: a feedback loop, not a slowdown. Why it exists: admission ignored the KV-budget math. What breaks: throughput graphs look busy while users wait, because recompute counts as work. Fix with admission control, not hardware.

1. Coordinated omission in benchmarks. A load generator that waits for a slow response before sending the next request hides queueing. Real users do not wait politely, so the measured p99 looks fine while real p99 is terrible.

Detect: compare generator-send timestamps against a fixed schedule. *Fix:* use a generator that sends on a fixed schedule regardless of response times.

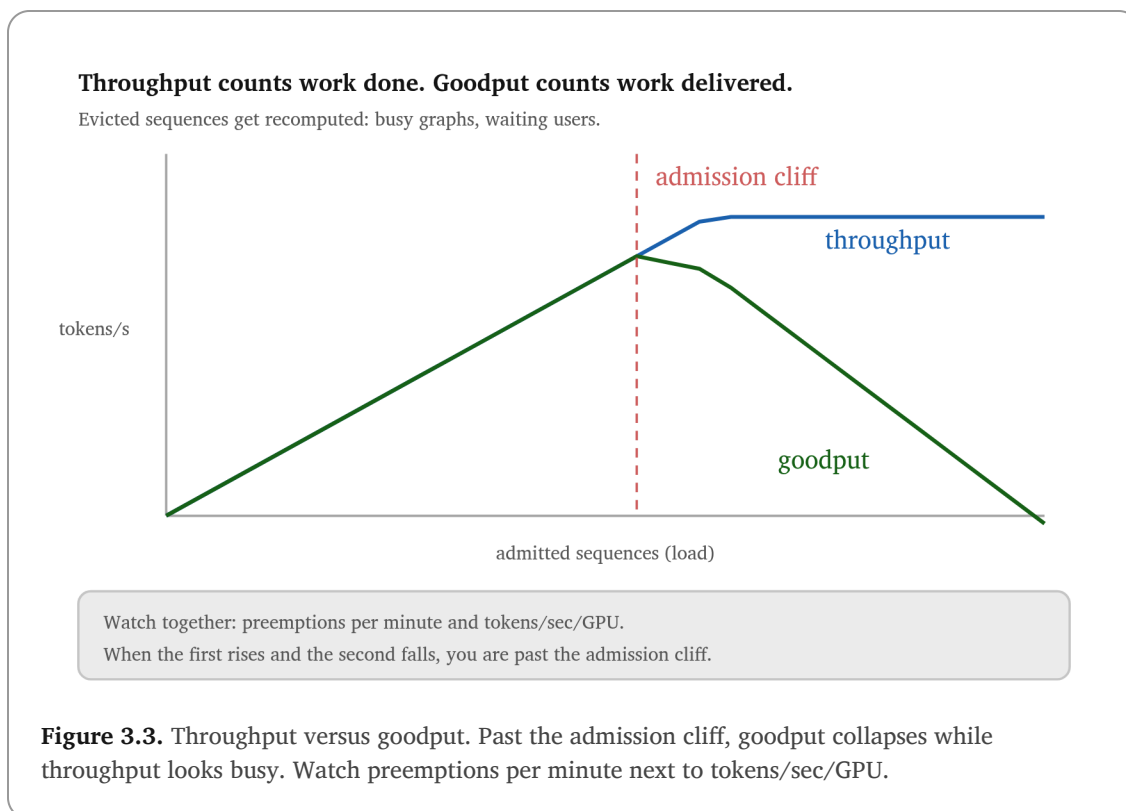
2. The KV-cache OOM cliff (OOM: out of memory). Max sequences x max context x bytes/token exceeds the KV budget. You scheduled the OOM at config time; traffic did not cause it.

Detect: do the section 2.2 arithmetic before choosing max-sequences. *Fix:* lower admission, shorten max context, or quantize the KV cache. After any OOM, capture block-manager stats and queue state *before* restarting. Restarting destroys the evidence.

3. Prefix-cache poisoning. One extra whitespace in the system prompt template zeroes out prefix-cache hits overnight. TTFT jumps, nothing else changes. *Detect:* watch cache hits per thousand requests, segmented by template version. *Fix:* pin template versions; treat prompt templates as code.

4. Confusing throughput with goodput. Throughput counts tokens computed. **Goodput** counts tokens that reach users. Recompute after preemption burns FLOPs for zero new tokens, so throughput can look healthy while goodput collapses.

Detect: track preemptions per minute next to tokens/sec/GPU. *Fix:* admission control, applied early. Better to queue requests than to admit and evict them.



5. Perplexity-only quantization eval. Perplexity barely moves while math and code accuracy fall off a cliff. *Detect:* the eval harness must include task metrics (reasoning, code, long-context retrieval) on traffic-like data. *Fix:* fix calibration (distribution match, more samples, per-channel scales on outlier layers), not the bit-width.

Trap 6: stale calibration scales after fine-tuning. Fine-tunes shift weight distributions. Scales are not portable across checkpoints. *Detect:* re-run the eval harness after every fine-tune, merge, or distillation. *Fix:* re-calibrate. Every time.

7. Symmetric K/V quantization. Applying one scheme to both keys and values gets the worst of both worlds: wasted scales on values, crushed channels on keys. Keys per-channel, values per-token, always. Section 2.6 shows the 8.4x damage difference.

8. The static-batching straggler tax. One long request holds the whole batch hostage; short requests pay the longest request's time. *Detect:* p99 latency far above p50 on mixed traffic. *Fix:* continuous batching. This should never happen in 2026.

9. Slow autoscaling killing spot savings. Spot GPUs sit idle while reserved GPUs queue, and the team blames preemptions. Preemptions cost ~5% of spot-served requests as re-tries. Slow scale-up costs far more.

Detect: compare spot-fleet utilization against reserved-fleet queue depth during ramps.

Fix: tune the autoscaler first, argue about preemption tolerance second.

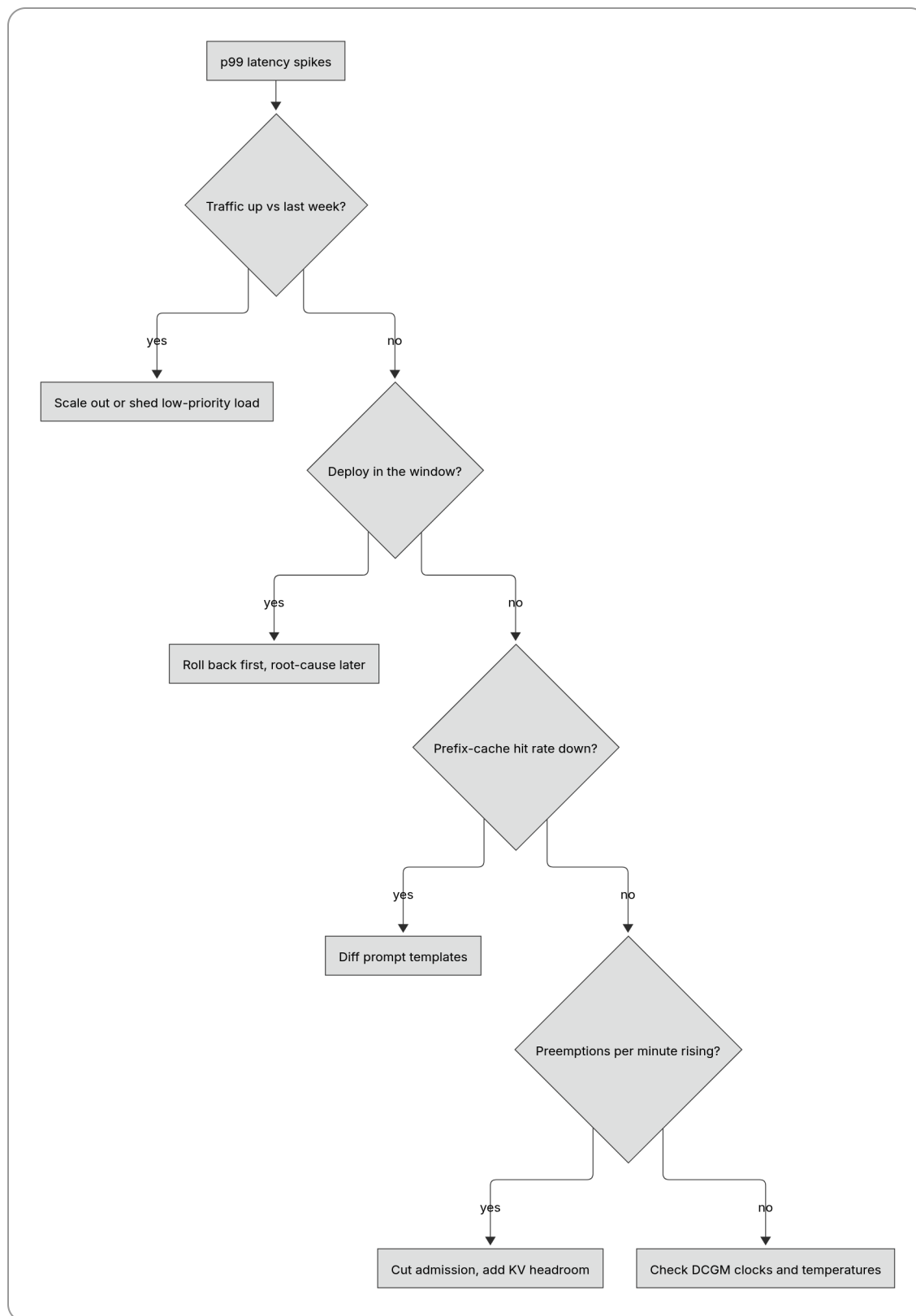
10. Shadow traffic without a comparator, or without a budget. Logs nobody reads are an expensive way to feel careful, and a 100% mirror doubles the GPU bill for the shadow period. *Detect:* ask who reads the comparator output and what the shadow week costs. *Fix:* sample 10% at first, write promotion gates before starting, shut the shadow down the day the decision is made.

11. The over-admission preemption storm. Someone raises max-concurrency "to use the GPUs better." Throughput per GPU falls 40%, preemptions jump 20x. Evicted sequences re-compute, which consumes the FLOPs that would have served new tokens, which keeps the queue long, which keeps pressure high. The storm feeds itself.

Detect: preemptions per minute spiking after a config change. *Fix:* revert the concurrency increase; set admission from the KV-budget math in section 2.2.

12. Disaggregating without measuring both sides. The team splits prefill and decode because "interference is bad" but never measures the transfer tax. A 32k-prompt transfer over IB costs ~215 ms; if the interference it removed was 50 ms, the split loses. *Detect:* run the transfer gate in section 2.7 with your link speeds. *Fix:* keep pools IB-local, or stay shared with chunked prefill.

Two incident rules that prevent most of the traps above from becoming outages. **Roll back before root-causing:** if a deploy correlates with the spike, reverting restores users in minutes; the post-mortem can take days. **Change one thing at a time:** two changes during an incident produce arguments, not knowledge. The third trap treats the symptom with more GPUs when the cause is a config (admission, chunk size, cache key). GPUs are expensive; configs are free.



How to read this diagram

1. Start at the top with traffic. Most spikes are demand, not defects. Compare RPS (requests per second) and tokens/sec against the same hour last week before touching anything.
2. A deploy in the window means roll back first. Pride has no place in incident response.
3. Cache hit rate is the quiet killer: a template change can zero prefix hits overnight with no other metric moving.
4. Rising preemptions mean the scheduler evicts sequences: cut admission from the KV-budget math.
5. If all of those are clean, check GPU health: a thermally throttling card (hot, clocks down) poisons every batch it touches. DCGM catches it in seconds.

Next: Section 4, rapid-fire questions with worked answers: roofline classification, KV capacity, cost math.

Twelve ways serving stacks fail: signature and fix**1. Coordinated omission**

signature: generator waits; real p99 terrible
fix: compare generator-send times

2. KV-cache OOM cliff

signature: OOM scheduled at config time, not caused by traffic
fix: do the 2.2 arithmetic first

3. Prefix-cache poisoning

signature: one whitespace zeroes prefix hits overnight
fix: pin template versions

4. Throughput vs goodput

signature: preemption burns FLOPs for zero new tokens
fix: track preemptions per minute

5. Perplexity-only quant eval

signature: perplexity flat while math falls off a cliff
fix: task metrics on traffic-like data

6. Stale calibration scales

signature: scales not portable across checkpoints
fix: re-calibrate after every fine-tune

7. Symmetric K/V quant

signature: one scheme for both: worst of both worlds
fix: keys per-channel, values per-token

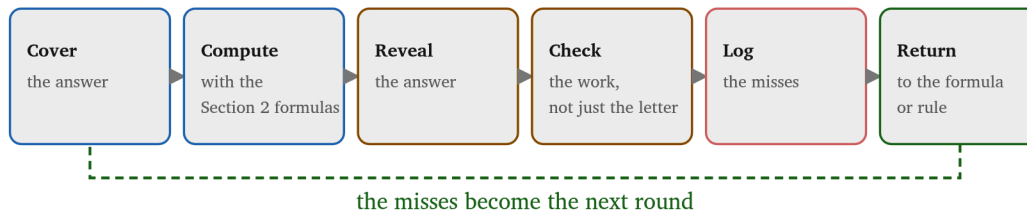
8. Static-batching stragglers

signature: one long request holds the batch hostage
fix: continuous batching

Figure 3.1. Twelve ways serving stacks fail: signature and fix. Each trap shows its signature first (what the failure looks like in the numbers) and the fix second.

4. Rapid-fire Q&A

Time to compute under pressure. The questions below are arithmetic or diagnosis, answered with the Section 2 formulas.

The rapid-fire loop: how to use Part 4

Time to compute under pressure. Every question is arithmetic or diagnosis, answered with the Section 2 formulas.

Figure 4.1. The rapid-fire loop: how to use Part 4. Time to compute under pressure: every question is arithmetic or diagnosis, answered with the Section 2 formulas. The misses become the next round.

Q1. Decode at batch 1 on an H100-class GPU uses roughly what fraction of peak BF16 compute, and what is the binding constraint?

- A. ~50%, compute-bound by attention FLOPs
- B. ~0.3%, memory-bandwidth-bound by weight reads
- C. ~10%, latency-bound by kernel launch overhead
- D. ~90%, compute-bound by the large batch

Answer: B. Each decode step reads the full 140 GB weight matrix to produce one token. 140 GFLOP against 140 GB moved gives ~1 FLOP/byte, far below the ~295 FLOP/byte ridge point. Attainable performance is ~3.35 TFLOPS out of 989, about 0.34% of peak. This is why batching, not faster math, is the first throughput lever.

A is wrong: attention FLOPs are small next to the weight reads. C is wrong: the 41.8 ms weight-read time dwarfs launch overhead. D is wrong: batch 1 is the opposite of a large batch; utilization is the problem.

Q2. A 70B-class model with GQA (8 KV heads) serves 32k-context requests. Roughly how much KV cache does each sequence need in BF16? What does that imply for batch size on a GPU with 10 GB of KV headroom?

- A. ~320 MB; about 30 sequences fit
- B. ~2.5 GB; about 4 sequences fit
- C. ~10 GB; about 1 sequence fits
- D. ~40 GB; it does not fit at all

Answer: C. $32,768 \text{ tokens} \times 320 \text{ KiB/token} = \sim 10 \text{ GB}$ per sequence. With 10 GB of headroom, exactly one such sequence fits: batch size 1, and throughput collapses. This is the long-context batch-size tax, and it is why KV-cache quantization and disaggregated decode pools exist.

A is the 1k-context figure. B is the 8k figure. D is the 128k figure.

Q3. The H100's BF16 ridge point is 295 FLOP/byte. A kernel has arithmetic intensity 55.6 FLOP/byte. Which roof binds, and what is the attainable performance?

- A. Compute roof; 989 TFLOPS
- B. Memory roof; $3.35 \times 55.6 = \sim 186 \text{ TFLOPS}$
- C. Neither; the kernel is latency-bound
- D. Memory roof; 3.35 TFLOPS

Answer: B. Intensity 55.6 is below the ridge 295, so the memory roof binds.

Attainable = bandwidth x intensity = $3.35 \text{ TB/s} \times 55.6 = \sim 186 \text{ TFLOPS}$. This is exactly the batch-64 decode case. A would need intensity above 295. D forgets to multiply by intensity.

Q4. Batch 64 decode has $\sim 55\times$ the throughput of batch 1, but the step time barely changes (48 ms vs 42 ms). Why?

- A. The GPU runs 55x faster at larger batches
- B. Batching amortizes the weight reads: one 140 GB read serves 64 tokens instead of 1
- C. KV cache reads shrink with batch size
- D. Prefill gets faster with batching

Answer: B. Every decode step reads the full weight matrix once regardless of batch size. At batch 64, that one read produces 64 tokens, so the bytes-per-token fall 64x while the step time stays roughly flat. Batching does not speed up any single request; it amortizes the weight reads across requests. C is backwards: KV reads grow with batch. D is irrelevant to decode step time.

Q5. Prefill of 2k tokens takes 0.29 s; one decode step takes 0.042 s. Per token, prefill is roughly 300x cheaper than decode. Why?

- A. Prefill uses fewer FLOPs per token
- B. Prefill reuses each weight read across all 2048 prompt tokens, so it sits on the compute roof instead of the memory slope
- C. Decode does more layers per token
- D. Prefill skips the KV cache

Answer: B. Prefill processes all prompt tokens in parallel. One pass over the weights serves 2048 tokens, pushing intensity to ~ 2033 FLOP/byte, past the ridge into compute-bound territory. Decode reads the same weights for a single token (intensity 1). Same weights, wildly different reuse.

C is wrong: both run all layers. D is wrong: prefill writes the KV cache.

Q6. Two models have the same 70B parameter count. One uses full multi-head attention (64 KV heads), the other GQA (8 KV heads). At 32k context, which serves cheaper and roughly by how much on KV memory?

- A. Same cost; KV heads do not affect serving
- B. The GQA model; ~8x less KV cache per token (320 KiB vs 2.5 MiB)
- C. The MHA model; more heads means better batching
- D. The GQA model; ~2x less

Answer: B. KV bytes per token = 2 x layers x KV-heads x head-dim x bytes. Cutting heads 64 -> 8 cuts the cache 8x: 320 KiB/token vs ~2.5 MiB/token. At 32k context that is 10 GB vs 80 GB per sequence. KV bytes per token matters as much as parameter count when comparing models for serving.

Q7. Your fleet is Hopper-class GPUs and decode throughput is the bottleneck. Which quantization choice is the best default, and why?

- A. GPTQ 4-bit weights, for maximum memory saving
- B. FP8 W8A8, near-zero accuracy cost with native hardware support
- C. GGUF 2-bit, for the smallest checkpoint
- D. No quantization; Hopper is fast enough already

Answer: B. On Hopper, FP8 is natively supported: W8A8 halves memory traffic with near-zero accuracy loss on most models, and calibration is light (static) or unnecessary (dynamic per-token). It attacks exactly the decode bottleneck, which is bytes per step.

A saves more memory but needs fused kernels for full speed and a careful calibration story. On Hopper, FP8 gets most of the win with less risk. C targets CPU/edge with real accuracy cost. D ignores the physics: decode is memory-bandwidth-bound on any GPU.

Q8. A 4-bit quantized model shows unchanged perplexity but a 6-point drop on math tasks. What is the most likely cause, and the fix?

- A. The GPU is too slow; use a faster GPU
- B. Perplexity-only evaluation hid the damage; investigate starting with calibration-data match and per-channel scales on outlier layers
- C. The batch size is wrong; increase it
- D. 4-bit is unusable; abandon quantization

Answer: B. Perplexity barely moves while reasoning degrades: this is the classic calibration failure. Start with the calibration set (does it match deployment distribution?), check sample count (reseed sensitivity means too few), then per-channel scales on outlier layers, then re-evaluate on task metrics. A is wrong: hardware speed does not change accuracy. C is wrong: batch size affects throughput, not task accuracy. D is wrong: 4-bit works well with correct calibration; the method is fine, the process failed.

Q9. In KV-cache quantization, keys use per-channel scales and values use per-token scales. Why this asymmetry?

- A. It is arbitrary; either layout works the same
- B. Keys carry persistent per-channel outliers (attention sinks), so per-token scales would stretch to cover the outlier and crush every normal value in that token. Values vary more token-to-token, so per-token is the natural and cheaper grouping
- C. Values are larger than keys
- D. Per-channel is always better for everything

Answer: B. Key outliers sit in fixed channels across all tokens. One scale per channel confines each outlier to its own range. Values have no such structure. They differ most from token to token, and a new token's scale is final the moment it arrives, which suits a streaming cache. Verified in section 2.6: per-token keys give 8.4x worse quant-channel error and erase 41% of meaningful values to zero.

Q10. FP8 has two flavors: E4M3 (range ± 448) and E5M2 (range $\pm 57,344$). Which do weights and activations use, and what goes wrong on overflow in each?

- A. E5M2 for weights; overflow saturates safely in both
- B. E4M3 for weights and activations (precision); E4M3 overflow saturates to 448 and silently biases sums, while E5M2 overflow becomes inf and poisons downstream math
- C. E4M3 for gradients only
- D. They are interchangeable

Answer: B. E4M3's 3 mantissa bits give the best precision of any 8-bit float, which is what weights and activations need. But E4M3 has no infinities: overflow saturates, silently biasing sums. E5M2 has infinities: overflow becomes inf, which poisons everything downstream. The format choice matters less than block-wise scaling, but the overflow behaviors must be known.

Q11. For FP8 serving on Hopper, when do you use dynamic per-token scales vs static calibrated scales?

- A. Always dynamic; calibration is obsolete
- B. Always static; dynamic never works
- C. Dynamic to try FP8 quickly with no prep; static (calibrated offline) for production, since dynamic per-step scale computation costs speed
- D. It makes no measurable difference

Answer: C. Dynamic FP8 (`--quantization fp8`) needs no calibration and is the fast path to trying FP8. But the per-step min/max computation costs speed, so production uses static scales calibrated offline (llm-compressor), with no runtime overhead. Benchmarking dynamic FP8 against static and concluding "FP8 is slow" measures the wrong mode.

Q12. A tensor has one huge outlier. Per-tensor FP8 scaling erases 100% of normal values to zero; block-128 scaling erases 3.2%. What is the mechanism?

- A. Smaller blocks use more bits per value
- B. Each block gets its own scale from its own max. This confines the outlier's oversized scale to its own block, and the other 31 blocks keep tight scales
- C. Block scaling skips the outlier
- D. Per-tensor scaling is a bug

Answer: B. $\text{Scale} = \text{block_amax} / 448$. The outlier sets a huge scale only for its own block; the other blocks' scales come from their own small maxima, preserving precision. Per-tensor scaling lets one value's scale zero out the whole tensor. The cost of block-128 is 32 scales for 4096 values: negligible overhead for the accuracy.

Q13. Under heavy load, an engine's preemption rate climbs sharply. What does this signal, and what is the first knob to turn?

- A. The GPUs are too slow; buy faster GPUs
- B. KV cache pressure: the scheduler evicts sequences and recomputes them, wasting work. First check max concurrent sequences and per-sequence context; reduce admission or add KV-cache quantization
- C. Traffic saturates the network; add bandwidth
- D. Prefill chunk size is too small; increase it

Answer: B. Preemption means the KV pool filled and the scheduler started evicting sequences, whose recompute burns FLOPs for zero new tokens. It is a capacity signal, not a speed signal. The fixes are admission control (fewer concurrent sequences), shorter contexts, or KV-cache quantization to fit more.

A is wrong: faster GPUs do not add KV capacity. C is wrong: preemption is GPU-local memory pressure. D is wrong: larger prefill chunks make steps longer and pressure worse.

Q14. Agentic traffic shares a 4k-token system prompt across thousands of requests. Which engine feature attacks this best, and how?

- A. TensorRT-LLM's engine build, which compiles the prompt away
- B. SGLang's RadixAttention with cache-aware scheduling. The engine computes the shared prefix once, stores it in the radix tree, and routes new requests to the worker holding it, skipping prefill
- C. Bigger batches, which amortize the prompt
- D. FP8 quantization, which halves the prompt bytes

Answer: B. RadixAttention stores the shared prefix once and shares it across requests; cache-aware routing sends each request to the worker already holding it. The 4k-token prefill happens once per worker, not once per request: a large TTFT and cost win. A is wrong: the engine build compiles the model, not per-request prompts. C is wrong: batching amortizes weight reads, not prompt compute. D is wrong: FP8 halves bytes but the prefill compute still happens per request.

Q15. What does chunked prefill protect, and what is the tuning knob?

- A. It protects TTFT; the knob is batch size
- B. It protects TPOT smoothness. A whole long prefill would stall running decodes, so the engine splits the prompt into fixed-size chunks interleaved one per step. The chunk size is the knob (bigger chunks: lower TTFT, lumpier TPOT)
- C. It protects KV memory; the knob is context length
- D. It is only for benchmarks

Answer: B. Without chunking, one big prefill is a long compute chunk that stalls every running decode, spiking TPOT. Interleaving fixed-size chunks keeps each step short. The chunk size trades TTFT against TPOT smoothness directly, so tune it against both budgets.

Q16. Your fleet is NVIDIA-only and you want maximum perf-per-dollar, accepting slower iteration. Your traffic is agentic with long shared prefixes. Which engine, and why might you still start elsewhere?

- A. TGI, for HF-native simplicity
- B. TensorRT-LLM for max NVIDIA perf-per-dollar (earliest FP8/FP4 kernels, hand-tuned plugins). But start with vLLM or SGLang for iteration speed and prefix sharing, then move hot paths to TRT-LLM once the design is stable
- C. vLLM, always, no exceptions
- D. Whichever has the nicest docs

Answer: B. TensorRT-LLM wins on raw NVIDIA perf-per-dollar but charges an engine build step per model/GPU, which slows iteration. SGLang's RadixAttention is the best answer for shared-prefix agentic traffic. The pragmatic path: iterate fast on vLLM/SGLang, promote the stable hot path to TRT-LLM. Engine choice follows traffic shape and fleet, not brand loyalty.

Q17. A 70B-class model serves 32k-token prompts. The team considers disaggregated serving with prefill and decode in different racks connected by InfiniBand (~50 GB/s effective). What does the transfer math say?

- A. Transfer is $\sim 10 \text{ GB} / 50 \text{ GB/s} = \sim 215 \text{ ms}$ per request, likely blowing a tight TTFT budget. Keep the pools interconnect-local or stay shared
- B. Transfer is free because RDMA has zero overhead
- C. Transfer is $\sim 10 \text{ MB}$, negligible at any scale
- D. Disaggregation always wins regardless of transfer cost

Answer: A. $32,768 \text{ tokens} \times 320 \text{ KiB} = \sim 10.7 \text{ GB}$ of KV cache. At 50 GB/s that is $\sim 215 \text{ ms}$ added to every request's TTFT, before prefill compute. Unless the TTFT budget tolerates it, this deployment loses to a shared pool.

RDMA avoids CPU copies but the bytes still take 215 ms on the wire (B is wrong). C is off by 1000x. D ignores the measured tax.

Q18. After disaggregating, the prefill pool idles at 30% utilization while the decode pool queues. What is wrong, and what is the fix?

- A. The model is too small; use a bigger model
- B. The pool ratio does not match the traffic mix: too many prefill GPUs for the actual prompt/output ratio. Re-run the sizer with measured traffic and rebalance; put each pool on its own autoscaler
- C. The transfer link is too fast; slow it down
- D. Nothing; idle prefill GPUs are normal

Answer: B. Size the two pools from the traffic mix independently. An idle prefill pool with a queueing decode pool means the ratio assumed longer prompts (or fewer outputs) than reality. Measure the real prompt/output distribution, resize, and autoscale each pool separately since the two phases scale differently. Disaggregation should avoid exactly this waste: 30% idle on expensive GPUs.

Q19. State the disaggregation decision rule: when do you split, and what two numbers must you compare?

- A. Always split; compare nothing
- B. Split when at least two of: tight TTFT SLO with interference as the measured cause, prompts 8k + , decode starved by prefill's KV usage, fast interconnect. Then verify: transfer under 25% of the TTFT budget AND transfer cheaper than the interference it removes
- C. Split when prompts are short
- D. Split when one pool would idle

Answer: B. The default is a shared pool with chunked prefill; disaggregation is the optimization you graduate to with measurements. Short prompts (C) make the transfer tax dominate. An idling pool (D) is a reason *not* to split. The two comparisons are the whole decision: budget fit and interference-vs-transfer.

Q20. A fleet's unit cost is \$2.00 per 1M tokens at 50% utilization. Throughput per GPU does not change. What is the unit cost at 80% utilization, and where should you invest?

- A. \$3.20; invest in faster GPUs
- B. \$1.25; utilization work (autoscaling, traffic shaping, pool rightsizing) pays more than most kernel work
- C. \$2.00; utilization does not affect unit cost
- D. \$1.00; unit cost halves whenever utilization rises

Answer: B. Unit cost scales inversely with utilization: $\$2.00 \times 0.50/0.80 = \1.25 . A 30-point utilization gain cut cost 37% with zero kernel changes. This is why capacity planning sits beside batching and quantization as a first-class lever. A inverts the direction. D misstates the proportion.

Q21. Spot GPUs are 65% cheaper but the provider can preempt them. Which workload split is sound, and why?

- A. Everything on spot; preemptions never matter
- B. Baseline load on reserved (sized for 10th-percentile load), peaks on spot, with graceful degradation (queue, shed, or cascade to a smaller model) when spot vanishes
- C. Everything on reserved; spot is never worth the complexity
- D. Decode on spot, prefill on reserved, because decode is stateless

Answer: B. The baseline is always there, so it belongs on capacity that never disappears. Peaks are variable, so they belong on cheap variable capacity. Graceful degradation bounds the damage when spot goes away, and the effective discount (~63%) survives realistic retry costs. D inverts the logic: decode workers hold in-flight KV state, so they are the *less* preemptible-friendly phase; prefill is short and cheap to redo.

Q22. A cascade serves 80% of traffic on a \$0.22/1M small model and escalates 20% to a \$1.11/1M big model. It uses a \$0.05/1M judge gate. What is the average cost per 1M tokens, and what is the escalation tax?

- A. \$0.40; escalated requests pay only the big model
- B. \$0.49; every request pays small + gate, and escalated requests pay the big model on top (small + gate + big)
- C. \$1.11; cascades never save money
- D. \$0.22; the big model is free

Answer: B. $0.22 + 0.05 + 0.20 \times 1.11 = \$0.49/1M$, still 2.3x cheaper than big-model-only. The escalation tax: escalated requests always pay twice (they ran the small model too). A (\$0.40) is the shortcut that forgets the small-model cost on escalated requests. Track the escalation rate like a budget: at 50% escalation the cost math nearly collapses and even p50 latency hits the big model.

Q23. Raw GPU cost for a 2,000-token request is \$0.0022. The fleet runs at 15% utilization and the task has a 70% success rate. What is the cost per successful outcome, and what does each layer mean?

- A. \$0.0022; raw cost is all that matters
- B. \$0.021; utilization divides by 0.15 (idle GPUs still bill) and success rate divides by 0.70 (each good answer carries 1.43 attempts)
- C. \$0.015; success rate does not matter
- D. \$0.21; utilization multiplies cost by 10

Answer: B. $\$0.0022 / (0.15 \times 0.70) = \0.021 . The raw number said two-tenths of a cent; the honest number says two cents, 10x higher. Utilization is usually the biggest lever (a scheduling problem, not a hardware problem), and the success layer is why a "cheaper" model that hallucinates more can cost more per good answer.

Q24. You shadow a cheaper quantized candidate for a week: 350k mirrored requests at \$0.014/request vs a \$0.020 baseline. Weekly savings after promotion would be \$2,100. What is the breakeven, and what must be true before you promote?

- A. 2.3 weeks; promotion gates (quality within 0.5 pts, p99 within 110%, cost at/below baseline, error bump under 0.5 pts) written before the shadow started, all passing, then a gradual ramp with rollback triggers
- B. Immediate; promote on day one
- C. Never; shadow traffic is free so there is nothing to break even on
- D. 2.3 weeks; promote if the mean latency looks good

Answer: A. \$4,900 shadow spend / \$2,100 per week = ~2.3 weeks, after which the shadow week was free. Write the gates before the shadow starts, or you will always find a story where the candidate wins. Compare tails (p99), not means: a candidate can match mean latency and still double your p99. C is wrong: at 100% mirroring the shadow fleet costs as much as production.

Q25. p99 latency doubles within an hour. RPS is flat, no deploy went out, cache hit rate is steady, preemptions are near zero. DCGM shows one GPU 65 C above its peers with clocks 30% below spec. Diagnosis and action?

- A. Scheduler bug; restart the engine
- B. Thermal throttling on a failing card: the hot GPU slows its steps and poisons batch latency. Drain the node, check cooling, RMA the card if the pattern repeats
- C. Traffic surge; add GPUs
- D. KV pressure; enable KV quantization

Answer: B. The tree rules out traffic, deploy, cache, and scheduler. A lone hot GPU with reduced clocks is thermal throttling, often a failing fan or blocked airflow. Because decode steps run in lockstep across the batch, one slow GPU slows every request it touches.

A is wrong: nothing points at the scheduler. C is wrong: RPS is flat. D is wrong: preemptions near zero means KV pressure is not the cause.

Q26. After enabling a higher max-concurrency setting, throughput per GPU falls 40% and preemptions per minute jump 20x. What happened, and what is the correct response?

- A. The GPUs got slower; replace them
- B. A preemption storm: admission exceeds KV capacity, sequences get evicted and re-computed, and recompute burns the FLOPs that would have served tokens. Revert the concurrency increase; set admission from the KV-budget math
- C. Network congestion is the cause; add bandwidth
- D. This is normal; higher concurrency always costs throughput

Answer: B. This is the textbook preemption storm: more admitted sequences than KV blocks, evictions, recompute waste, falling goodput. The storm feeds itself: recompute consumes the FLOPs that would serve new tokens, keeping the queue long and pressure high. The fix is admission control grounded in the capacity math, not more hardware. D is wrong: higher concurrency helps until KV capacity binds; past that point it destroys throughput. The cliff is the signal you crossed the line.

Q27. A 70B BF16 model OOMs on a single 80 GB GPU at load time. The team wants to tune the scheduler. What do you check first, second, third?

- A. Scheduler flags, then batch size, then the model
- B. Weights first (140 GB cannot fit on 80 GB; this is not a tuning problem), then the KV budget (max sequences x max context x 320 KiB/token must fit with headroom), then fragmentation vs leak (memory growth over 24h: a climbing sawtooth is a leak, flat-then-death is fragmentation or a burst)
- C. The network, then the disk, then the scheduler
- D. Add more GPUs immediately

Answer: B. OOMs are arithmetic, done in order. Weights: $70B \times 2 \text{ bytes} = 140 \text{ GB}$. This needs two 80 GB GPUs with tensor parallelism before anything else.

Also verify quantization actually applied (a config typo can silently load FP16). Then the KV budget from section 2.2. Then the time dimension: growth over hours means leak (unreleased blocks, cached graphs), sudden death on a flat line means fragmentation or a burst.

A tunes the wrong layer. D spends money on an arithmetic problem.

Q28. One 128k-output request halves a pool's throughput. Why, and what are the two defenses?

- A. It uses more FLOPs; buy faster GPUs
- B. Long outputs hold KV blocks for the whole generation, starving the batch of KV memory. Defenses: cap output length, or route long generations to a separate pool
- C. It is a network problem; add bandwidth
- D. Nothing can be done; this is normal

Answer: B. KV blocks are held for the entire generation, so one very long output occupies memory that could have served many short requests. It is a capacity problem, not a speed problem. Caps on output length protect the main pool; a separate pool for long generations isolates the damage. This is the same batch-size tax as section 2.2, wearing an output-length hat.

Q29. An MoE layer shows one expert receiving 40% of tokens while seven others idle. The imbalance factor is ~ 3.5 . What does this cost, and what are the fixes?

- A. Nothing; experts are independent
- B. The hottest rank sets the layer's latency: the layer waits $\sim 3.5x$ longer on the hot expert. Fixes: EPLB (redundant copies of hot experts on extra ranks), SGLang Waterfill (+1.5-4.7%), LPLB (+0.8-7.3%). Alert threshold: one expert above 40% of tokens; healthy is uniform within $\pm 20\%$
- C. Add more experts
- D. Reduce top-k to 1

Answer: B. Under expert parallelism, tokens dispatch over the network to expert shards; the overloaded rank is the straggler for the whole layer. The fix is not faster experts. Spread the hot expert's load. Monitor per-expert token counts in production (vLLM exposes them via Prometheus) and alert when the imbalance factor crosses ~ 2 .

Q30. You must serve 100 fine-tuned variants of a 7B model. Compare one deployment per tenant vs multi-tenant LoRA, and name the operational risks of the LoRA path.

A. Same cost either way

B. Naive: $100 \times 14 \text{ GB} = 1,400 \text{ GB}$ (eighteen 80 GB GPUs before KV cache). Multi-tenant LoRA: one 14 GB base + $100 \times 16 \text{ MB}$ adapters = $\sim 15.6 \text{ GB}$, $\sim 90\times$ less, with heterogeneous batching sharing the base matmul. Risks: cold-adapter fetch latency (warm adapters first), adapter versioning, and the runtime-load endpoint, which must stay trusted-admin-only since it loads weight files from a request-supplied path

C. LoRA is always worse

D. Just buy 100 GPUs

Answer: B. The win is architectural: per-tenant cost drops from gigabytes to megabytes, and heterogeneous batching keeps utilization high. The remaining hard problems are operational, not mathematical: size `--max-loras` from the tenant traffic distribution (hot adapters on GPU, rest spilled to CPU under LRU), warm before production, and lock down dynamic loading.

Q31. Your benchmark reports p99 latency of 400 ms, but users complain. The load generator sends the next request only after the previous response arrives. What is wrong, and how do you fix the harness?

A. Nothing; users are wrong

B. Coordinated omission: the generator waits politely for slow responses, so it never queues and never measures queueing. Real users arrive on their own schedule. Fix: send on a fixed schedule (requests arriving on a random open schedule, like real users) regardless of response times, and warm up 50+ requests before measuring

C. The model is too slow

D. p99 is the wrong metric; use the mean

Answer: B. A closed-loop generator hides the very queueing you want to measure. When the system slows, the generator slows its sends to match, and the latency histogram looks rosy. An open-loop fixed-schedule generator keeps arriving during slow-downs, which is what real traffic does. Also warm up before measuring (cold kernels and caches poison early samples), split by traffic class, and watch p999 too.

Q32. Tokens/sec/GPU looks healthy, but users see retries and slow streams. Preemptions per minute are high. What metric confusion is at work?

- A. None; the stack is fine
- B. Throughput vs goodput: the engine computes many tokens (throughput) but pre-emption-recompute burns FLOPs for zero new tokens, so few reach users (goodput). Watch goodput = tokens reaching users / tokens computed, and treat rising preemptions as the scheduler's distress signal
- C. The GPUs are too slow
- D. Goodput is not a real metric

Answer: B. Throughput counts work done; goodput counts work delivered. Evicted sequences get recomputed, which looks like activity on the throughput graph while users wait. The pair to watch together is preemptions per minute and tokens/sec/GPU. When the first rises and the second falls, you are past the admission cliff. Fix with admission control, not hardware.

Q33. Overnight, TTFT p99 jumps 3x. Throughput, error rate, and preemptions are all flat. What is the most likely cause, and which metric confirms it?

- A. Traffic surge; check RPS
- B. Prefix-cache poisoning: a prompt-template change (even one whitespace) zeroed prefix-cache hits. Confirm with cache hits per thousand requests, segmented by template version
- C. GPU failure; check DCGM
- D. Quantization regression; check evals

Answer: B. Prefill dominates TTFT, and prefix caching skips exactly the prefill. When hits drop to zero, every request pays full prefill again: TTFT jumps while nothing else moves. Throughput is flat because the GPUs do the same total work, just more prefill and less useful decode. The confirming metric is prefix-cache hit rate segmented by template version. The fix: treat prompt templates as versioned code.

Next: Section 5, the thirty numbers and rules that matter most, on one page.

5. Cheat sheet: the 30 numbers and rules that matter most

The whole pack on one page: hardware peaks, KV math, cost anchors, and the rule one-liners.

1. H100 SXM: 989 TFLOPS BF16 dense, 1979 TFLOPS FP8, 3.35 TB/s HBM.
2. Ridge points: 295 FLOP/byte (BF16), 591 FLOP/byte (FP8). Below the ridge you are memory-bound; the fix is fewer bytes, not more FLOPS.
3. Decode batch 1 (70B BF16): intensity 1, ~ 3.35 TFLOPS attainable = 0.34% of peak, 41.8 ms/step, ~ 24 tok/s.
4. Decode batch 64: intensity 55.6, ~ 186 TFLOPS, 48 ms for 64 tokens = $\sim 1,333$ tok/s. Batching amortizes weight reads; it never speeds up one request.
5. Prefill 2k tokens: compute-bound, 0.29 s, $\sim 300\times$ cheaper per token than decode.
6. KV bytes/token = $2 \times \text{layers} \times \text{KV-heads} \times \text{head-dim} \times \text{bytes}$. 70B GQA BF16: 320 KiB/token.
7. Per-sequence KV: 1k $\rightarrow$ 320 MB; 8k $\rightarrow$ 2.56 GB; 32k $\rightarrow$ 10.2 GB; 128k $\rightarrow$ 41 GB.
8. GQA 8 heads vs MHA 64 heads: 8x less KV cache. Architecture is a serving decision.
9. TTFT = queue wait + prefill. TPOT = average decode step. Tight TTFT SLO: under ~ 1 s.
10. Always report p99 and p999, never the mean alone.
11. $\$/1\text{M tokens} = \text{price} \times \text{GPUs} / (\text{tok/s} \times 3600 \times \text{utilization}) \times 1\text{e}6$.
12. 70B on 2x H100 at $\$3/\text{hr}$: $\$1.11/1\text{M}$ at 100% util, $\$1.85$ at 60%, $\$2.22$ at 50%.
13. Batch ladder: $b=1 \rightarrow \$13.89$; $b=8 \rightarrow \$2.38$; $b=64 \rightarrow \$1.11$ per 1M. Same GPUs.
14. Cost per successful outcome = raw / (utilization x success rate). $\$0.0022$ raw $\rightarrow$ $\$0.021$ honest.
15. Utilization: 50% $\rightarrow$ 80% cuts unit cost 37% ($\$2.00 \rightarrow \1.25). Capacity planning is a first-class lever.
16. FP8: E4M3 (+/-448, weights/activations) vs E5M2 (+/-57,344, range). W8A8 needs compute 8.9+. 2x memory, up to 1.6x throughput.
17. Static FP8 (calibrated) ships; dynamic FP8 (per-token scales) is for trying. Never judge FP8's speed on the dynamic mode.
18. Block-128 FP8 scaling: one outlier erases 100% of values per-tensor, 3.2% block-128, 0.8% block-32.
19. KIVI: keys per-channel, values per-token, ~ 128 -token fp16 residual window. int8 KV $\sim$ lossless; int4 for long context.
20. GPTQ/AWQ 4-bit: pre-Hopper max memory saving; pick by engine kernel support and benchmark both.
21. Chunked prefill protects TPOT; chunk size trades TTFT vs smoothness.

22. Rising preemptions = scheduler distress. Recompute burns FLOPs for zero tokens. Cut admission.
23. Prefix-cache-aware routing (SGLang RadixAttention): shared prompts computed once per worker.
24. Disaggregate when 2 of 4 hold: tight TTFT with measured interference, prompts 8k+, decode starved of KV, fast interconnect. Default: shared + chunked prefill.
25. Transfer sizes (70B): 8k -> ~6 ms NVLink / ~54 ms IB; 32k -> ~24 ms NVLink / ~215 ms IB. Transfer must beat the interference it removes.
26. Spot 50-70% cheaper (~63% effective). Baseline on reserved (10th percentile); peaks on spot. Prefill is more spot-friendly than decode.
27. Cascade: every request pays small + gate; escalated pay big too. 80/20 -> \$0.44-0.49/1M vs \$1.11 (2.3-2.5x). Budget the escalation rate.
28. Shadow: gates written before (quality +/-0.5, p99 <= 110%, cost <=, errors + 0.5pp); ramp 1/10/50/100; breakeven ~2.3 weeks in the worked example.
29. MoE: 671B total / 37B active. Hot-expert alert: one expert > 40% of tokens. Fixes: EPLB, Waterfill, LPLB.
30. LoRA: rank-16 7B adapter ~16 MB. 100 tenants: 1,400 GB naive vs 15.6 GB shared (~90x).

Last verified: September 2026. Hardware numbers (H100 specs, cloud prices) are public spec-sheet and listing values from September 2026. Treat them as illustrative, not quotes. Every worked number in this pack is an illustrative example. Every Python snippet ran, and its printed output passed verification. Redo the math with your own hardware prices and measured throughput before spending money.

UNVERIFIED: engine feature details and flags evolve quickly. Verify scheduler flags, quantization support, and kernel availability against each engine's current documentation before relying on them in production. **UNVERIFIED:** all tokens/second, latency, and utilization figures are illustrative worked examples, not measured benchmarks.

NOTE: per-token KV figures elsewhere in the curriculum's economics supplement use a different illustrative model shape. This pack's canonical figure is the derived 320 KiB/token for the 70B-class GQA model (2 x 80 x 8 x 128 x 2 bytes).

Where to go next

The next pack in the sequence is [Crash: Distributed Training](#). This pack served the model. The next one answers the question this pack leaves open: how did the model get trained that big in the first place? Parallelism strategy, collectives, checkpointing, MFU, and the TPU/XLA stack.

WATCH AND GO DEEPER

Two videos that show the machinery in motion: how inference actually scales, and how vLLM's PagedAttention kills the memory waste.

How LLM Inference Actually Scales (Codemia).

Chapters build the full serving stack step by step. The chapters cover: prefill vs decode, the KV cache, MHA/GQA/MQA, static to continuous batching, the fragmentation wall, PagedAttention, prefix caching, speculative decoding. Then: routing, scheduling, and failure modes. This machine could not fetch the transcript (bot-blocked), so this rests on the author's own chapter list. It maps onto this pack's sections one for one.

Fast LLM Serving with vLLM and PagedAttention (Anyscale).

The paper's own author (Woosuk Kwon, UC Berkeley) walks through why serving was slow. He covers the KV cache, block tables killing fragmentation, and the 24x throughput claim. Transcript verified: this is the talk the PagedAttention figures above come from.

GO DEEPER

vLLM documentation

The engine's own docs: configuration, quantization options, and the scheduler knobs this pack's rules tune.

PagedAttention paper (SOSP 2023)

The paper behind the block-table figures above: how non-contiguous KV blocks kill fragmentation.

Orca paper (SOSP 2022)

The paper that introduced continuous (in-flight) batching for LLM serving.